

# Aula 1 - Motivação Inicial

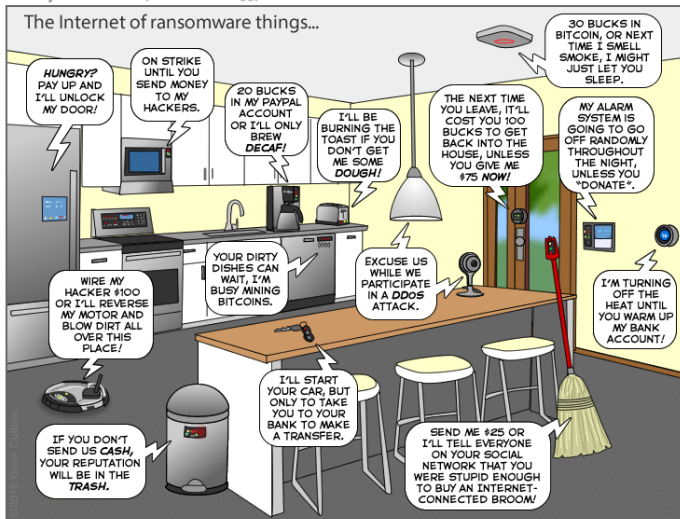
Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação  
Universidade Federal do Rio de Janeiro  
gabrielaquino@ic.ufrj.br

Compilado em:  
April 3, 2026

# Preocupações da atualidade

The Joy of Tech™ by Nitrozac & Snaggy



You can help us keep the comics coming by becoming a patron!  
[www.patreon.com/joyoftech](http://www.patreon.com/joyoftech)

[joyoftech.com](http://joyoftech.com)

## Ativo valioso

A informação é o ativo mais valioso para uma organização ou para uma pessoa.

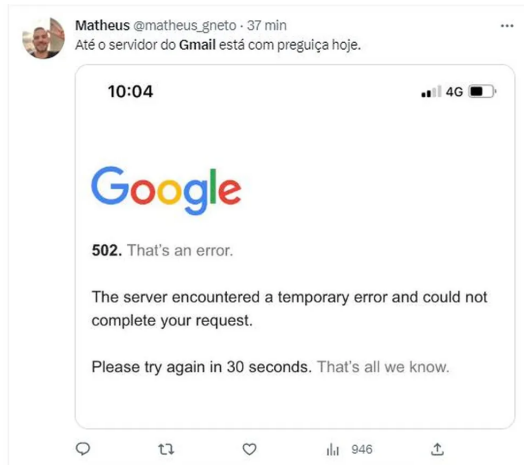
## Por que?

- **tomada de decisão.**
- **vantagem competitiva.**
- **ativo financeiro, estratégico ou pessoal.**

## Impactos da perda ou comprometimento

- Prejuízos financeiros.
- Danos à reputação.
- Risco jurídico.

# Preocupações da atualidade



Link: Notícia

# Preocupações da atualidade

yahoo/news

## Microsoft outage: CrowdStrike security update impacts airports, hospitals, banks around the world

A CrowdStrike security update was automatically installed across Microsoft servers early Friday morning.



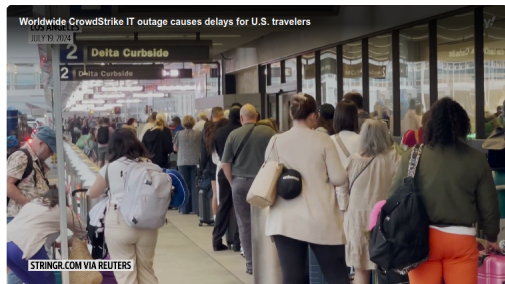
Katie Mather, Reporter

Updated Fri, July 19, 2024 at 2:33 PM GMT-3

6 min read



023k



Link: Notícia

## Megavazamento de dados de 223 milhões de brasileiros: o que se sabe e o que falta saber

Número é maior do que a população do país, estimada em 212 milhões, porque inclui dados de falecidos. Informações expostas incluem CPF, nome, sexo e data de nascimento, além de uma tabela com dados de veículos e uma lista com CNPJs. Origem dos dados ainda é desconhecida.

Por G1

28/01/2021 18h34 · Atualizado há 4 anos

### Quais dados vazaram?

**Foram dois vazamentos:**

Um deles tinha **223 milhões de números de CPF**, acompanhado de informações como **nome, sexo e data de nascimento**, além de uma tabela com **dados de veículos e uma lista com CNPJs** (Cadastro Nacional de Pessoas Jurídicas). Essas informações circulam na Internet de forma gratuita.

O outro incluía, além dos 223 milhões de CPFs, informações sobre **escolaridade, benefícios do INSS e programas sociais (como o Bolsa Família)**, **renda**, entre outras informações. Esse está sendo vendido por criminosos.

Juntos, os dois vazamentos continham:

- Dados básicos relativos ao CPF (nome, data de nascimento e endereço).
- Endereços.
- Fotos de rosto.
- Score de crédito (que diz se é bom pagador), renda, cheques sem fundo e outras informações financeiras.
- Imposto de renda de pessoa física.
- Dados cadastrais de serviços de telefonia.
- Escolaridade.
- Benefícios do INSS.
- Dados relativos a servidores públicos.
- Informações do LinkedIn.

Link: Notícia

# Caso real brasileiro

## Caso Real no Brasil — 2025

- Em julho de 2025, um ataque hacker contra a empresa C&M Software resultou no desvio de cerca de **R\$541 milhões** de uma instituição financeira.
- C&M Software provia a infraestrutura técnica para que seus clientes (bancos pequenos, fintechs, cooperativas de crédito, etc.) conseguissem se conectar ao Sistema de Pagamentos Instantâneos (SPI) e ao Sistema de Pagamentos Brasileiro (SPB) do Banco Central.

## Repercussão na imprensa:

[Link 1](#) ; [Link 2](#) ; [Link 3](#) ; [link 4](#)

## O que esse caso demonstra

- Ataque que usou o fator humano ( "**insiders** confiáveis") - Fator humano é crítico.
- Uma única falha de segurança pode resultar em **prejuízos bilionários**.

# Quanto Vale a Informação?

## Ideia central

O esforço despendido para proteger a informação depende diretamente do **valor dessa informação**.

## Reflexão

Se a informação for **crítica** para o negócio ou para a vida das pessoas, a proteção deve ser proporcionalmente **robusta**.

# Quanto Vale a Informação?

## Princípio Econômico da Segurança da Informação

O custo para se proteger contra uma ameaça deve ser **menor** que o custo de recuperação caso a ameaça se concretize. RFC 2196

Link: RFC 2196 - Site Security Handbook

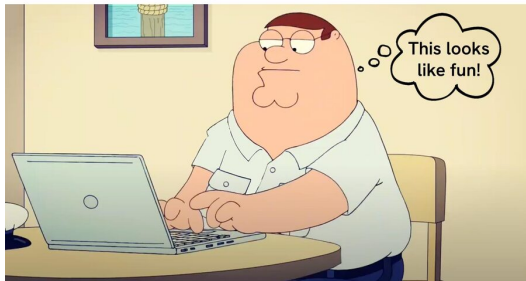
## Reflexão

Investir em segurança não é gasto — é evitar perdas maiores.

Só que... no mundo real...



When you think you've closed all the security gaps,  
but your employee clicks on a phishing email!



# Aula 2 - Conceitos Básicos de Segurança da Informação

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação  
Universidade Federal do Rio de Janeiro  
gabrielaquino@ic.ufrj.br

Compilado em:  
April 3, 2026

## Computer Security (COMPUSEC) (NISTIR 7298 - [Link](#)):

Medidas e controles que garantem a **confidencialidade**, **integridade** e **disponibilidade** dos ativos de sistemas de informação, incluindo hardware, software, firmware e as informações processadas, armazenadas e comunicadas.

- **Confidencialidade dos dados:** Garante que informações privadas ou confidenciais não sejam disponibilizadas ou divulgadas a indivíduos não autorizados.
- **Privacidade:** Garante que os indivíduos tenham controle ou influência sobre quais informações relacionadas a eles podem ser coletadas, armazenadas e divulgadas, e por quem.

**Confidencialidade** é o sigilo de informações ou recursos.

A necessidade de manter informações em segredo surge do uso de computadores em áreas sensíveis - Ex: Governo, indústria e outras áreas.

- Instituições militares e civis do governo aplicam o princípio do "**necessário saber**".
- Empresas protegem projetos proprietários contra concorrência desleal.
- Todas as organizações mantêm registros pessoais em sigilo.

Mecanismos de **controle de acesso** são essenciais para garantir a confidencialidade.

Um dos principais mecanismos é a **criptografia**, que embaralha os dados para torná-los incompreensíveis sem a chave correta.

## Exemplo:

- Um sistema cifra uma declaração de imposto de renda.
- Apenas quem possui a **chave criptográfica** pode decifrá-la e acessar os dados.
- Porém, se a chave for exposta durante seu uso, a confidencialidade é comprometida.

*A proteção da chave é tão crítica quanto a proteção da própria informação.*

**Confidencialidade** não se aplica apenas ao conteúdo dos dados, mas também à **existência da informação**.

## **Exemplos:**

- Saber que uma pesquisa foi realizada pode ser mais revelador que seu resultado.
- O conhecimento de que houve perseguição por uma agência pode ser mais crítico que os detalhes dessa perseguição.

## **Ocultação de Recursos:**

- Organizações podem esconder sua infraestrutura para evitar abusos ou exposição.

*Mecanismos de controle de acesso podem ocultar tanto dados quanto sua própria existência.*

# VeraCrypt: Criptografia de Disco

**VeraCrypt** é uma ferramenta de código aberto para **criptografia de dados em disco**, derivada do TrueCrypt.

## Principais Características:

- Criptografia em tempo real (*on-the-fly encryption*);
- Suporte a algoritmos robustos: AES, Serpent, Twofish, e combinações;
- Possibilidade de criar volumes ocultos para negar a existência dos dados (*plausible deniability*);
- Funciona em Windows, Linux e macOS.

## Casos de Uso:

- Proteger HDs externos, pendrives e partições do sistema;
- Garantir confidencialidade em notebooks e dispositivos móveis;
- Armazenar backups criptografados de forma segura.

Link: [Caso do uso TrueCrypt](#)

## Proteção de Dados Médicos com Criptografia

- **Cenário Completo:**
  - Rede de 5 hospitais compartilhando exames de imagem (raio-X, ressonâncias)
  - Dados classificados como máxima sensibilidade
- **Solução Multicamadas:**
  1. Criptografia AES-256 em repouso (armazenamento)
  2. TLS 1.3 com certificados X.509 em trânsito
  3. Chaves protegidas por HSM (Hardware Security Module)
- **Exemplo Detalhado:** Fluxo de um laudo de câncer de mama:

| <b>Etapas</b>        | <b>Proteção Aplicada</b>        |
|----------------------|---------------------------------|
| Digitação médico     | Criptografia de disco BitLocker |
| Transmissão de dados | Tunelamento VPN + TLS           |
| Armazenamento nuvem  | Criptografia e compliance       |

## Hierarquia de Acesso às Notas

- **Arquitetura:** Controle por papéis (RBAC) com políticas granulares

| Perfil      | Próprias Notas | Notas da Turma | Todas Notas |
|-------------|----------------|----------------|-------------|
| Aluno       | ✓              | –              | –           |
| Professor   | ✓              | ✓              | –           |
| Coordenação | ✓              | ✓              | ✓           |

- **Exemplo Prático:**
  - Aluno João visualiza apenas suas notas em Álgebra
  - Professora Maria vê todas as notas da turma de Cálculo II
  - Coordenador Pedro acessa boletins de todos os alunos do curso

- **Integridade dos dados:** Garante que informações e programas sejam modificados apenas de maneira autorizada e específica.
- **Integridade do sistema:** Garante que o sistema desempenhe sua função prevista de forma íntegra, livre de manipulações não autorizadas, intencionais ou acidentais.

# Integridade: Confiança nos Dados

**Integridade** refere-se à **confiabilidade dos dados ou recursos**, geralmente no sentido de prevenir modificações indevidas ou não autorizadas.

## Dimensões da Integridade:

- **Integridade dos dados:** garante que o conteúdo da informação não foi alterado indevidamente.
- **Integridade da origem (autenticidade):** assegura que a fonte dos dados é confiável e legítima.

A integridade da origem afeta a **credibilidade**, que é fundamental para o funcionamento adequado de um sistema.

## Exemplo:

Um jornal publica uma informação obtida por vazamento da Casa Branca, mas atribui a fonte incorretamente.

⇒ **Integridade dos dados** preservada; **integridade da origem** comprometida.

# Mecanismos de Integridade

Os mecanismos de integridade dividem-se em duas classes: **mecanismos de prevenção** e **mecanismos de detecção**.

**Mecanismos de prevenção** mantêm a integridade dos dados bloqueando tentativas não autorizadas de alterar os dados ou modificá-los de formas não autorizadas.

Distinção importante:

- Tentativa de alterar dados sem autorização (ex: invasor tentando modificar dados).
- Tentativa de alterar dados de formas não autorizadas por usuários autorizados (ex: contador desviando dinheiro para conta no exterior).

**Mecanismos de detecção** indicam que a integridade dos dados foi comprometida.

- Analisar eventos do sistema (ações de usuários ou do sistema) para detectar problemas.
- Analisar os próprios dados para verificar se as restrições esperadas ainda são válidas.

Podem indicar a causa específica da violação ou apenas reportar que o arquivo está corrompido.

# Integridade vs. Confidencialidade

Trabalhar com integridade é muito diferente de trabalhar com confidencialidade.

Na confidencialidade, os dados ou foram comprometidos ou não.

Já a integridade inclui tanto a **correção** quanto a **confiabilidade** dos dados.

Fatores que afetam a integridade dos dados:

- A origem dos dados (como e de quem foram obtidos).
- O quão bem os dados foram protegidos antes de chegar à máquina atual.
- O quão bem os dados são protegidos na máquina atual.

Avaliar a integridade é difícil, pois depende de suposições sobre a origem dos dados e a confiança nessa origem — aspectos fundamentais de segurança que muitas vezes são negligenciados.

# Verificando a Integridade de Arquivos Baixados

Ao baixar arquivos da internet, é comum verificar sua integridade para garantir que:

- O arquivo não foi corrompido durante a transferência.
- O arquivo não foi adulterado por um atacante.

**Como fazer isso?** Usando funções de hash criptográficas como:

- MD5 – não recomendado hoje em dia mas muito usado.
- SHA-256 – mais recomendado atualmente.

**Exemplo no terminal:**

```
$ sha256sum arquivo.iso
```

```
$ md5sum arquivo.iso
```

Compare o valor obtido com o publicado pelo site oficial.

# Sistema de acadêmico com garantia de integridade: Como garantir

- **Registro Oficial**

- Cada professor registra notas com login único e seguro
- Toda alteração precisa ser aprovada pela coordenação
- Data e hora são registradas automaticamente

- **Proteção contra Alterações**

- Notas lançadas não podem ser apagadas
- Mudanças criam um novo registro (como versão de documento)
- Relatórios mensais são conferidos pela diretoria

## Exemplo Prático: Professor dá nota para um aluno

1. O sistema registra:

- Quem lançou: Professora Ana
- Quando: 15/03/2023 às 14:30
- Versão do registro: 001

2. Se precisar corrigir, cria-se um novo registro (versão 002)

# Aula 3 - Conceitos Básicos de Segurança da Informação

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação  
Universidade Federal do Rio de Janeiro  
gabrielaquino@ic.ufrj.br

Compilado em:  
April 3, 2026

- Garante que os sistemas **funcionem** corretamente e que os serviços **não sejam negados** aos usuários autorizados.
- Não adianta ter dados íntegros e protegidos se o sistema está fora do ar.

**Disponibilidade** refere-se à capacidade de acessar a informação ou o recurso desejado quando necessário.

- É essencial para a confiabilidade e o funcionamento do sistema.
- Um sistema indisponível é tão inútil quanto um sistema inexistente.
- Ataques como **DoS (Denial of Service)** exploram falhas de disponibilidade.
- Projetos de sistema geralmente assumem um *modelo estatístico* de uso.
- Um atacante pode distorcer esse modelo (ex: tráfego de rede excessivo) para causar falhas.

*Se o sistema não foi projetado para o cenário de ataque, os mecanismos de proteção podem falhar.*

# Preocupação com Disponibilidade

| Availability % | Availability (Nines) | Downtime per Year   | Downtime per Month  | Downtime per Week   | Downtime per Day    |
|----------------|----------------------|---------------------|---------------------|---------------------|---------------------|
| 90%            | One nine             | 36.53 days          | 73.05 hours         | 16.80 hours         | 2.40 hours          |
| 99%            | Two nines            | 3.65 days           | 7.31 hours          | 1.68 hours          | 14.40 minutes       |
| 99.9%          | Three nines          | 8.77 hours          | 43.83 minutes       | 10.08 minutes       | 1.44 minutes        |
| 99.99%         | Four nines           | 52.60 minutes       | 4.38 minutes        | 1.01 minutes        | 8.64 seconds        |
| 99.999%        | Five nines           | 5.26 minutes        | 26.30 seconds       | 6.05 seconds        | 864.00 milliseconds |
| 99.9999%       | Six nines            | 31.56 seconds       | 2.63 seconds        | 604.80 milliseconds | 86.40 milliseconds  |
| 99.99999%      | Seven nines          | 3.16 seconds        | 262.98 milliseconds | 60.48 milliseconds  | 8.64 milliseconds   |
| 99.999999%     | Eight nines          | 315.58 milliseconds | 26.30 milliseconds  | 6.05 milliseconds   | 864.00 microseconds |
| 99.9999999%    | Nine nines           | 31.56 milliseconds  | 2.63 milliseconds   | 604.80 microseconds | 86.40 microseconds  |

[Link: Referência - Oracle](#)

# De onde vêm os dados de disponibilidade?

Seus cálculos de disponibilidade serão tão bons quanto os dados neles inseridos.

Precisamos de dados sobre interrupções e tempo de inatividade.

Os dados de tempo de serviço e tempo de inatividade podem ser coletados de diversas fontes, incluindo:

- Teste de ping
- Software de monitoramento
- Tickets de suporte técnico / call center
- Relatórios de TI quando ocorrem incidentes de interrupção
- Sistemas de Gestão de Informações e Eventos de Segurança (SIEMs)
- Análise de IA de arquivos de log e outras entradas e saídas do sistema

# Disponibilidade - Exemplo de Ataque DDoS

## O Problema:

- **Cenário:** Sistema de matrículas online da universidade
- **Quando:** Primeiro dia do período de matrículas
- **Sintoma:** Sistema extremamente lento ou fora do ar

## O Que Aconteceu?

- Ataque DDoS (sobrecarga de acessos falsos)
- +100.000 solicitações por segundo de bots
- Servidores não conseguiram atender alunos reais

## Soluções de prevenção e recuperação:

### • Prevenção:

- Filtro contra tráfego malicioso
- Limite de tentativas por IP

### • Recuperação:

- Servidores reserva automáticos
- Sistema prioritário para alunos cadastrados

# Dificuldade em Detectar Ataques de Negação de Serviço (DoS)

- Ataques que visam tornar recursos indisponíveis são chamados de **ataques de negação de serviço (DoS)**.
- São **difíceis de detectar**, pois é necessário distinguir entre:
  - Manipulação intencional de recursos ou do ambiente;
  - Padrões de uso incomuns, mas legítimos.
- Os modelos estatísticos usados para prever o uso normal:
  - Consideram eventos atípicos como parte da distribuição estatística;
  - Podem não identificar um ataque como anômalo.
- Em alguns cenários, o ataque pode nem parecer atípico.

Para detectar comportamentos estranhos na rede...

Precisamos ver os dados de funcionamento da rede

# O que é um fluxo (flow) no NetFlow?

Um *flow* é definido como uma sequência unidirecional de pacotes com propriedades comuns que passam por um dispositivo de rede. Esses fluxos coletados são exportados para um dispositivo externo, o *NetFlow collector*.

Um registro de fluxo inclui:

- Endereços IP, Contagem de pacotes e bytes
- *Timestamps*, Tipo de Serviço (ToS)
- Portas de aplicação, Interfaces de entrada e saída

Os dados exportados do NetFlow são usados para:

- Faturamento de ISPs
- Monitoramento e planejamento de capacidade
- Monitoramento e perfil de aplicações e usuários
- Análise de segurança
- Mineração de dados para fins de marketing

[Link: RFC 3954 - NetFlow versão 9](#)

# Dados da rede - O que é NetFlow?

- Protocolo de rede desenvolvido pela **Cisco Systems** para coletar metadados sobre o tráfego IP em roteadores, switches e hosts.
- Permite monitorar e obter conhecimento sobre o desempenho de aplicações e da rede.
- Fornece informações sobre:
  - Quantidade de tráfego;
  - Origem e destino do tráfego;
  - Caminhos utilizados.
- Estatísticas de fluxo ajudam no monitoramento, identificação de problemas e planejamento de upgrades.
- [Link: O que é o Netflow](#)
- [Link: Configurando o Netflow em roteadores CISCO](#)
- [Link: Softflowd no Ubuntu](#)

## Principais componentes:

### 1. Exportador de NetFlow:

- Agrega pacotes em fluxos e exporta registros via UDP.
- Identifica fluxos com base em IP, portas, protocolo e tipo de serviço.
- Exporta fluxos inativos ou encerrados (ex: flags TCP FIN/RST).

### 2. Dados da rede - Coletor NetFlow:

- Recebe registros agregados de exportadores.
- Pré-processa e armazena os dados.

### 3. Dados da rede - Analisador de NetFlow:

- Processa e analisa os registros coletados.
- Gera relatórios e alertas sobre largura de banda, padrões de tráfego e uso de aplicações.

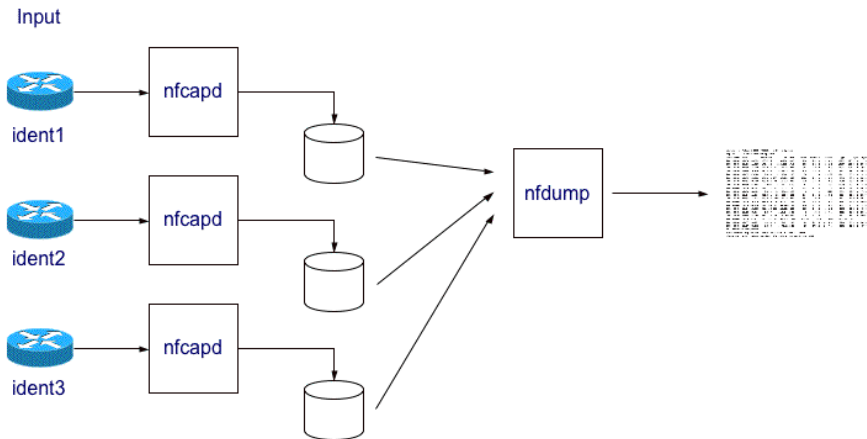
**NFDUMP:** Coleção de ferramentas para coleta e processamento de dados *NetFlow* via linha de comando. Faz parte do projeto **NfSen**.

- [Link: Github do projeto Nfdump](#)

## Principais ferramentas:

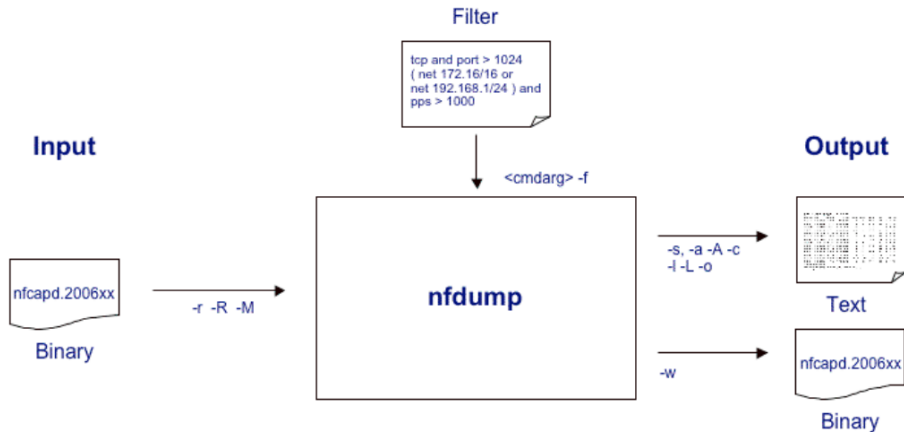
- **nfcapd** — Daemon que captura fluxos de rede (*NetFlow* v5, v7 e v9) e armazena em arquivos.
- **nfdump** — Lê e exibe dados *NetFlow* dos arquivos, semelhante ao *tcpdump*.
- **nfprofile** — Cria perfis de *NetFlow* com base em filtros.
- **nfreplay** — Reenvia dados de fluxo para outro host.
- **nfclean.pl** — Limpa dados antigos de forma periódica.
- **ft2nfdump** — Converte dados de outras ferramentas de fluxo para o formato *nfdump*.

# Dados da rede - Arquitetura NFDUMP



Link: [Ferramenta NFDUMP](#)

# Dados da rede - Como o NFDUMP funciona



[Link: User Documentation nfdump e NfSen](#)

**Front-end gráfico baseado na Web** para as ferramentas de NetFlow do NFDUMP.

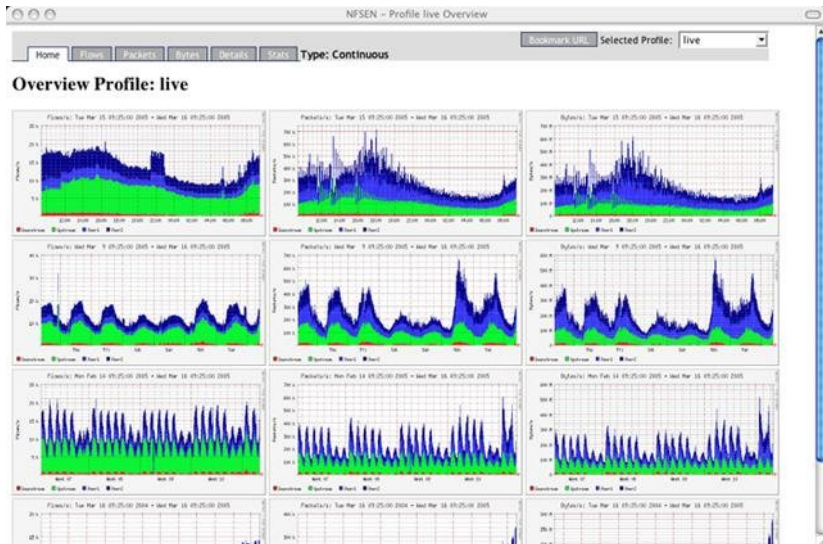
## **Funcionalidades:**

- Exibir e navegar facilmente pelos dados do NetFlow.
- Processar dados dentro de um período especificado.
- Criar histórico e perfis contínuos de tráfego.
- Definir alertas com base em várias condições.
- Desenvolver plugins personalizados para processar dados periodicamente.

*Disponível no SourceForge, sob licença BSD.*

- [Link: Sourceforge do NFSEN](#)

# Dados da rede - Front-End Nfsen



**ANÁLISE, DETECÇÃO E DESENVOLVIMENTO  
PARA PREVENÇÃO DE ANOMALIAS EM  
SISTEMAS DE COMUNICAÇÃO DE ALTO  
DESEMPENHO NO CIBERESPAÇO**

Layson Rodrigues da Costa (layson@cbpf.br)  
supervisora: Marita Maestrelli (marita@cbpf.br)

Jornada PCI-CBPF 2021/2022 Novembro de 2022

13:21 / 1:45:09

Link: [Apresentação no YouTube \(Começa em 13:21\)](#)

Link: [Arquivo .pdf da apresentação](#)

# Aula 4 - Conceitos Básicos de Segurança da Informação

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação  
Universidade Federal do Rio de Janeiro  
gabrielaquino@ic.ufrj.br

Compilado em:  
April 3, 2026

# Três pilares da Segurança da informação

Os três pilares da segurança da informação são:

## Confidencialidade

**Definição:** Proteção contra acesso não autorizado

**Ameaça:** Vazamento de dados

**Exemplo:** Sigilo de processos judiciais

## Integridade

**Definição:** Prevenção de alterações indevidas

**Ameaça:** Fraude em registros

**Exemplo:** Sistemas para eleições

## Disponibilidade

**Definição:** Acesso contínuo aos sistemas

**Ameaça:** Ataques DDoS

**Exemplo:** Plataforma durante ataques ou crises

Exemplo dos três pilares em um sistema:

- Sigilo das transações (Confidencialidade)
- Manutenção dos saldos (Integridade)
- Operação 24/7 (Disponibilidade)

## Extensão dos três pilares: Autenticidade e Não-Repúdio

- **Autenticidade e não-repúdio:** Garantia da origem legítima das informações e impossibilidade de negar participação em uma transação.
  - **Autenticidade:** Propriedade que garante que a origem de uma informação é legítima e verificável.
  - **Não-repúdio:** Capacidade de provar que uma ação ocorreu e que sua origem não pode ser negada pelas partes envolvidas.

# Como ter autenticidade mas não ter não-repúdio? - Exemplo 1

**Cenário:** Um usuário preenche um formulário em papel, marcando opções com um “X” e assinando no final.

**Autenticidade:** A assinatura garante que o formulário foi realmente assinado pelo usuário — **autenticidade**.

**Não-Repúdio:** Posteriormente, o usuário alega: “Eu não marquei estas opções”. Como não há registro eletrônico ou prova criptográfica do preenchimento das opções, não é possível contestar a alegação — **não-repúdio**.

**Como garantir não repúdio:**

- Uso de formulários digitais com marcação eletrônica vinculada à assinatura digital.
- Registro seguro de cada ação com timestamp, de forma auditável.

## Como ter autenticidade sem ter não-repúdio? - Exemplo 2

**Cenário:** Um funcionário envia um pedido de reembolso via e-mail corporativo.

**Autenticidade:** O e-mail vem de sua conta corporativa e o sistema confirma que ele é o remetente — **autenticidade**.

**Não-repúdio:** O funcionário depois alega: “Eu não enviei este pedido, alguém acessou minha conta”. Sem uma assinatura digital ou registro auditável adicional, a empresa não consegue provar de forma incontestável que ele realmente enviou o e-mail — **não-repúdio**.

**Como garantir não-repúdio:**

- Assinatura digital do e-mail ou do pedido.
- Logs seguros e auditáveis da ação com timestamp.

## **Autenticidade**

Propriedade de ser genuíno, verificável e confiável. Envolve ter confiança na validade de uma transmissão, mensagem ou origem da mensagem. Verifica se os usuários são quem dizem ser e se cada entrada recebida veio de uma fonte confiável.

Exemplo:

- Documento manuscrito: comparação das características de escrita com amostras verificadas.
- Informação eletrônica: uso de assinatura digital com criptografia de chave pública para verificar autoria e, possivelmente, a integridade.

## **Não-repúdio**

Capacidade de provar que um evento ou ação ocorreu e qual foi sua origem, evitando que as partes envolvidas neguem posteriormente.

- Prova de entrega para o remetente.
- Prova de identidade do remetente para o destinatário.

## Exemplo: Assinatura Digital via ITI (ICP-Brasil)

**Cenário:** Assina digital de documento utilizando o assinador do ITI ([Link](#)) Usando a Infraestrutura de Chaves Públicas (ICP) ([ICP-Brasil](#)).

- Certificado digital vinculado à identidade do usuário.
- Chave privada usada para assinar digitalmente o documento.
- Registro auditável da assinatura.
- Verificação automática de alterações posteriores ao documento assinado.

**Autenticidade:** O sistema verifica que a assinatura foi feita pelo titular do certificado digital ou pela conta *gov.br* — **autenticidade**.

**Não-Repúdio:** Assinatura feita com a chave privada do usuário e registrada no site do ITI (Assinador ITI), o usuário **não pode negar posteriormente** que assinou o documento — **não repúdio**.

**Integridade:** Se alguém tentar alterar o documento após a assinatura, o sistema detecta que ele foi modificado, garantindo que o conteúdo original permanece inalterado.

## Autenticidade

- Garante que a mensagem ou ação veio realmente de quem afirma tê-la realizado.
- Exemplos:
  - Login com senha ou biometria.
  - Certificado digital em sites HTTPS.
  - Assinatura digital para verificação do remetente.

## Não-Repúdio

- Impede que o autor de uma ação negue sua responsabilidade posteriormente.
- Exemplos:
  - Assinaturas digitais com certificado ICP-Brasil.
  - Registros assinados de transações financeiras.
  - Logs de auditoria assinados digitalmente.

# Extensão dos três pilares: Responsabilização (Accountability)

- **Definição de Accountability:** Segurança que exige que ações de uma entidade sejam rastreadas exclusivamente para ela.
- **Objetivos de Accountability:**
  - Suportar o **não repúdio**.
  - Prevenir ou desencorajar **comportamentos indesejados**
  - Promover **isolamento de falhas**.
  - **Detectar e prevenir de intrusões**.
  - Facilitar a **recuperação após ação** e apoio para ações legais.
- **Importância:**
  - Permite rastrear brechas de segurança até a parte responsável.
  - Registros de atividades são fundamentais em uma para análise forense.
  - Dá insumos para se resolver disputas.

# Extensão dos três pilares: Responsabilização (Accountability)

## Responsabilização (Accountability)

- Capacidade de atribuir ações a indivíduos ou entidades específicas.
- Capacidade de exigir que indivíduos respondam por suas ações.
- Envolve identificação, autenticação e registro de atividades.
- Inclui mecanismos de auditoria, rastreabilidade e sanções.
- Permite investigação de incidentes de segurança.
- Exemplo: logs assinados digitalmente, políticas claras de uso.

Ou seja, precisamos lidar com quem fez o quê e tratar de como lidamos com quem fez o quê — ou seja, as consequências. Em termos técnicos: Você pode ter responsabilização, mas a responsabilização é a ação que responsabiliza alguém se sabe quem agiu. (ex: os logs mostram quem fez, mas nada é feito com isso)

# Exemplo de Responsabilização no Linux/Unix

**Ferramenta:** auditd (Auditing Daemon)

**Objetivo:** Capturar e registrar ações de usuários no sistema, garantindo rastreabilidade e possibilidade de responsabilização.

**Como funciona:**

- Logs de acesso localizados em `/var/log/audit/audit.log` ou `/var/log/secure`.
- Registram login/logout, comandos executados e alterações em arquivos críticos.
- Cada ação é associada a um UID, horário e tipo de operação.

**Exemplo prático:**

- Um usuário cria ou modifica um arquivo no sistema.
- `auditd` registra o evento com:
  - Identificação do usuário (UID)
  - Hora e data da ação
  - Comando executado ou arquivo alterado

**Uso:** Auditoria de segurança, investigação de incidentes, cumprimento de políticas corporativas e rastreabilidade legal.

# Auditd - Adicionar regra

Para monitorar um arquivo `test_audit.txt` usando a tag `test_aula`:

## Comandos

```
sudo auditd
# Adiciona a regra para auditoria
sudo auditctl -w ~/test_audit.txt -p wa -k test_aula

# Verifica as regras ativas
sudo auditctl -l

# Executa alguma ação no arquivo...
bash -c "echo 'linha de teste' >> /home/$USER/teste_audit.txt"

# Consulta eventos pela tag
sudo ausearch -k test_aula
sudo ausearch -k test_aula --format text
```

# Auditd - Explicando os parâmetros

- `auditctl` — ferramenta para gerenciar regras no `auditd`.
- `-w` — caminho do arquivo ou diretório a ser monitorado.
- `-p` — permissões a auditar:
  - `r` — leitura
  - `w` — escrita
  - `x` — execução
  - `a` — alteração de atributos
- `-k` — chave (tag) para identificar os eventos, usada no `ausearch`.
- `ausearch -k` — busca no log do audit por eventos que contenham a chave especificada.

# Auditd - Interpretando a saída do ausearch

Ao executar:

```
ausearch -k test_aula
```

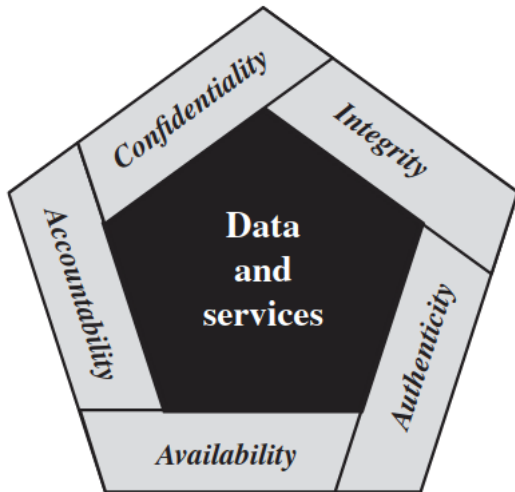
Na saída, é possível observar:

## Exemplo de trecho relevante

```
type=SYSCALL msg=audit(1692000000.123:420): \  
  arch=c000003e syscall=257 success=yes \  
  exit=3 a0=... auid=... uid=... comm="bash" exe="/usr/bin/bash" ...  
  proctitle=62617368(...)7874
```

- syscall=257 corresponde à chamada de sistema `openat`, usada para abrir/criar arquivos.
- proctitle é exibido em hexadecimal. Ao decodificar `62617368...`, obtemos: `bash -c "echo a >> test_audit.txt"`.
- Veja o usuário em `getent passwd "AUID"`

# Os 5 elementos da Segurança da Informação



**Figure 1.1 Essential Network and Computer Security Requirements**

# Aula 5 - Conceitos de criptografia e Criptografia Simétrica

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação  
Universidade Federal do Rio de Janeiro  
gabrielaquino@ic.ufrj.br

Compilado em:  
April 3, 2026

**Os sistemas criptográficos são caracterizados em três dimensões:**

**1. Tipo de operações:**

- **Substituição:** cada elemento do texto claro é mapeado em outro elemento.
- **Transposição:** os elementos do texto claro são rearranjados.
- A maioria dos sistemas combina ambos em várias etapas (sistemas de produto).

**2. Número de chaves:**

- **Encriptação simétrica:** mesma chave para emissor e receptor.
- **Encriptação assimétrica:** chaves diferentes (pública/privada).

**3. Modo de processamento:**

- **Cifra de bloco:** processa blocos inteiros de elementos.
- **Cifra de fluxo:** processa elementos continuamente, um a um.

# Encriptação Simétrica

- Também chamada de **encriptação convencional** ou **de chave única**.
- Era o único tipo em uso antes da década de 1970, quando surgiu a encriptação por chave pública.
- Continua sendo o muito utilizado até hoje.

## Definições iniciais:

- Texto claro (*plaintext*): mensagem original.
- Texto cifrado (*ciphertext*): mensagem codificada.
- Cifração (ou encriptação): processo de transformar texto claro em texto cifrado.
- Decifração (ou decríptação): processo de recuperar o texto claro a partir do texto cifrado.

- **Criptografia:** estudo e construção de esquemas de encriptação (as chamadas cifras ou sistemas criptográficos).
- **Criptoanálise:** técnicas para decifrar uma mensagem sem conhecimento prévio da chave ou do método usado (o famoso "quebrar o código").
- **Criptologia:** área mais ampla que engloba tanto a criptografia quanto a criptoanálise.

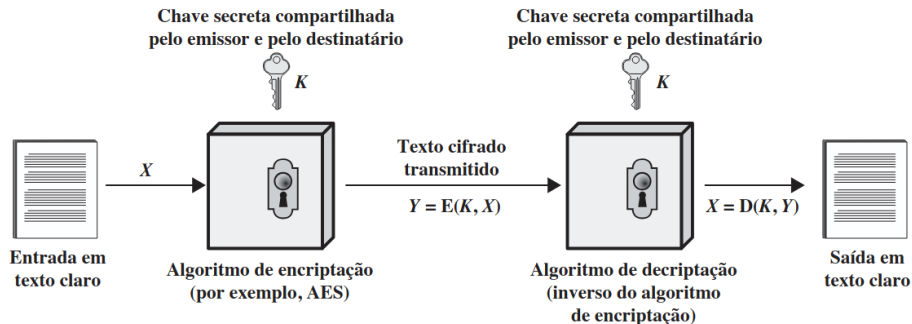
**Resumo:** Criptografia = criar códigos. Criptoanálise = quebrar códigos. Criptologia = o estudo de ambos.

Um esquema de encriptação simétrica possui cinco componentes:

- **Texto claro:** mensagem original e inteligível, entrada do algoritmo.
- **Algoritmo de encriptação:** aplica substituições e transformações no texto claro.
- **Chave secreta:** valor independente do texto claro e do algoritmo; define as transformações realizadas.
- **Texto cifrado:** saída do processo de encriptação, com aparência aleatória e ininteligível.
- **Algoritmo de decifração:** operação inversa à encriptação, recupera o texto claro a partir do texto cifrado e da chave.

# Esquema de Encriptação Simétrica

**Figura 2.1** Modelo simplificado da encriptação simétrica.



**Fluxo:** Texto Claro  $\xrightarrow{\text{Encriptação} + \text{Chave}}$  Texto Cifrado  $\xrightarrow{\text{Decriptação} + \text{Chave}}$  Texto Claro

# Requisitos para o Uso Seguro da Encriptação Simétrica

Para que a encriptação simétrica seja efetiva, dois requisitos fundamentais devem ser atendidos:

1. **Algoritmo forte:** mesmo conhecendo o algoritmo e tendo acesso a textos cifrados (com ou sem os respectivos textos claros), um oponente não deve ser capaz de descobrir a chave ou recuperar o texto claro.
2. **Chave protegida:** emissor e receptor devem compartilhar a chave secreta de forma segura e mantê-la em sigilo. Caso a chave seja comprometida, toda a comunicação associada poderá ser lida.

# Segurança na Encriptação Simétrica

- É impraticável decriptar uma mensagem apenas com o texto cifrado e o conhecimento do algoritmo.
- O segredo deve estar somente na **chave**, não no algoritmo.
- Essa característica torna a encriptação simétrica viável para uso generalizado.
- Como o algoritmo pode ser público, fabricantes desenvolvem **chips de baixo custo** com encriptação embutida, hoje comuns em diversos produtos.
- Portanto, o principal desafio de segurança é **manter o sigilo da chave**.

# Esquema de Encriptação Simétrica

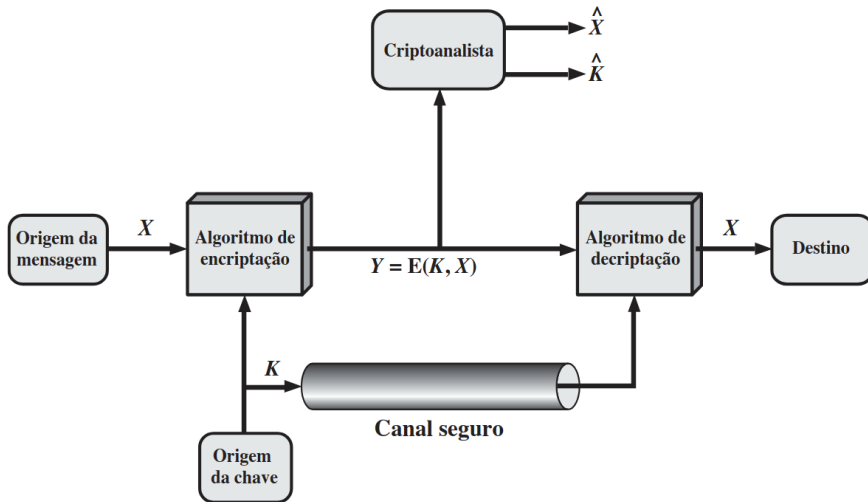
- Texto claro:  $X = [X_1, X_2, \dots, X_M]$
- Chave de encriptação:  $K = [K_1, K_2, \dots, K_J]$
- Texto cifrado:  $Y = E(K, X)$
- Decriptação:  $X = D(K, Y)$
- A chave  $K$  deve ser compartilhada com segurança entre origem e destino (diretamente ou por terceiro confiável).

## Visão do Oponente

- Oponente observa  $Y$ , mas não conhece  $K$ .
- Supõe-se que conhece os algoritmos  $E$  e  $D$ .
- Dois objetivos possíveis:
  - Recuperar o texto claro  $X$ , obtendo uma estimativa  $\hat{X}$ .
  - Recuperar a chave  $K$ , obtendo uma estimativa  $\hat{K}$  e assim ler mensagens futuras.

# Modelo de criptossistema simétrico

**Figura 2.2** Modelo de criptossistema simétrico.



# Criptanálise e Ataque por Força Bruta

O objetivo de um ataque é **recuperar a chave em uso**, não apenas o texto claro.

## Criptanálise

- Explora a natureza do algoritmo de encriptação.
- Pode usar conhecimento prévio sobre o texto claro ou pares texto claro–cifrado.
- Busca deduzir a chave ou recuperar mensagens específicas.

## Ataque por Força Bruta

- Testa todas as chaves possíveis no texto cifrado.
- Em média, é necessário tentar metade do espaço de chaves.
- Garante sucesso se houver tempo e poder computacional suficiente.

# Os tipos de Ataques à criptografia

A forma de atacar depende da **quantidade de informação disponível** para o atacante:

- **Apenas texto cifrado:** cenário mais difícil, exige análise estatística.
- **Algoritmo conhecido:** normalmente se assume que o adversário sabe qual cifra está sendo usada.
- **Ataque por força bruta:** testar todas as chaves possíveis.
  - Impraticável se o espaço de chaves for muito grande.
- **Conhecimento do tipo de texto claro:** facilita a análise (ex.: textos em inglês/francês, arquivos EXE, código fonte, planilhas contábeis).

# Ataque de Texto Claro Conhecido

O ataque apenas com texto cifrado é o mais fácil de ser defendido, pois o oponente tem a quantidade mínima de informação para trabalhar

**Caso:** O atacante possui uma ou mais mensagens em **texto claro** e suas correspondentes mensagens cifradas.

- Ele pode explorar padrões previsíveis no texto claro.
  - Arquivos com cabeçalhos fixos (ex. PDFs ou logs).
  - Mensagens financeiras ou contratos eletrônicos com banners ou formatos padronizados.
  - Protocolos de rede com campos fixos (ex.: pacotes com cabeçalhos conhecidos).
- Com essas informações, é possível **deduzir a chave** ou reduzir significativamente o espaço de chaves a ser testado.
- Normalmente, ataques de texto claro conhecido são mais efetivos contra cifras fracas ou mal projetadas.

# Ataque de Palavra Provável e Texto Claro Escolhido

## **Ataque de Palavra Provável:**

- Variante do ataque de texto claro conhecido.
- O atacante possui conhecimento parcial de partes da mensagem ou de palavras específicas.
- Exemplos:
  - Arquivos de contabilidade com cabeçalhos padronizados ou palavras-chave em posições fixas.
  - Código fonte com notas de direitos autorais em posições específicas.
- Permite ao atacante reduzir o espaço de chaves a testar ou deduzir padrões na cifra.

## **Ataque de Texto Claro Escolhido:**

- O atacante consegue que a origem encripte mensagens de sua escolha.
- Pode introduzir padrões específicos que ajudem a revelar a estrutura da chave.

- Baseia-se em limitações computacionais ou econômicas do atacante.
- Dois critérios principais:
  - **Custo:** quebrar a cifra é mais caro que o valor da informação.
  - **Tempo:** quebrar a cifra leva mais tempo do que a vida útil da informação.
- Objetivo: tornar a decifração inviável na prática, mesmo que teoricamente possível.
- Um esquema de encriptação é considerado computacionalmente seguro se um desses dois critérios for atendido.
  - **Problema:** é muito difícil estimar a quantidade de esforço exigido para criptoanalisar textos cifrados com sucesso.
- Ciptoanálise para esquemas de encriptação simétricos são projetadas para explorar o fato de que rastros da estrutura ou do padrão do texto claro podem sobreviver à encriptação e ser discerníveis no texto cifrado

**Definição:** Uma cifra de substituição simples usada por Júlio César. Cada letra do alfabeto é substituída por aquela que está **três posições à frente**.

**Exemplo:**

- Texto claro: meet me after the toga party
- Texto cifrado: PHHW PH DIWHU WKH WRJD SDUWB

**Alfabeto:**

|    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|
| a  | b  | c  | d  | e  | f  | g  | h  | i  | j  | k  | l  | m  |
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 |
| n  | o  | p  | q  | r  | s  | t  | u  | v  | w  | x  | y  | z  |
| 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 |

**Observação:** O alfabeto é circular, ou seja, após Z, volta-se para A.

# Tabela cifra de César

| CHAVE | PHHW | PH | DIWHU | WKH | WRJD | SDUWB |
|-------|------|----|-------|-----|------|-------|
| 1     | oggv | og | chvgt | vjg | vqic | rctva |
| 2     | nffu | nf | bgufs | uif | uphb | qbsuz |
| 3     | meet | me | after | the | toga | party |
| 4     | ldds | ld | zesdq | sgd | snfz | ozqsx |
| 5     | kccr | kc | ydrpc | rfc | rmey | nyprw |
| 6     | jbbq | jb | xcqbo | geb | qldx | mxoqv |
| 7     | iaap | ia | wbpan | pda | pkcw | lwnpu |
| 8     | hzzo | hz | vaozm | ocz | objv | kvmot |
| 9     | gyyn | gy | uznyl | nby | niau | julns |
| 10    | fxxm | fx | tymxk | max | mhzt | itkmr |
| 11    | ewwl | ew | sxlwj | lzw | lgys | hsjlq |
| 12    | dvvk | dv | rwkvi | kyv | kfxr | grikp |
| 13    | cuuj | cu | qvjuh | jxu | jewq | fqhjo |
| 14    | btti | bt | putg  | iwt | idvp | epgin |
| 15    | assh | as | othsf | hvs | hcuo | dofhm |
| 16    | zrrg | zr | nsgre | gur | gbtn | cnegl |
| 17    | yqqf | yq | mrfqd | ftq | fasm | bmdfk |
| 18    | xppe | xp | lqepc | esp | ezrl | alcej |
| 19    | wood | wo | kpdob | dro | dyqk | zkbdi |
| 20    | vnnc | vn | jocna | cqn | cxpj | yjach |
| 21    | ummb | um | inbmz | bpm | bwoi | xizbg |
| 22    | tlla | tl | hmaly | aol | avnh | whyaf |
| 23    | skkz | sk | glzkx | znk | zumg | vgxze |
| 24    | rjjy | rj | fkyjw | ymj | ytlf | ufwyd |
| 25    | qiix | qi | ejxiv | xli | xske | tevxc |

# Algoritmo Matemático da Cifra de César

**Cifra de César com deslocamento fixo (3 posições):**

$$C = E(3, p) = (p + 3) \mod 26$$

**Cifra de César geral com deslocamento  $k$ :**

$$C = E(k, p) = (p + k) \mod 26, \quad k \in \{1, 2, \dots, 25\}$$

**Decifragem:**

$$p = D(k, C) = (C - k) \mod 26$$

**Observações:**

- Cada letra do alfabeto é mapeada numericamente de 0 a 25.
- O operador  $\mod 26$  garante que o alfabeto seja circular (após Z vem A).
- Qualquer deslocamento  $k$  no intervalo de 1 a 25 gera uma cifra válida.
- $p$  representa cada letra do texto claro.

# Exemplo de Ataque por Palavra Provável - Cifra de César ( $k=6$ )

## Texto cifrado:

*G sotng igyg k asg igyg wak loig vkxzu jk uazxg igyg*

## Texto claro (César, deslocamento $k = 6$ ):

*A minha casa e uma casa que fica perto de outra casa*

**Observação:** A palavra "**casa**" aparece repetida várias vezes no texto cifrado (igyg).

## Ataque por palavra provável:

- Reconhecemos padrões repetidos (igyg) que correspondem à palavra "casa".
- Com isso, podemos deduzir o deslocamento usado ( $k = 6$ ).
- Após determinar a chave, podemos decifrar o restante do texto.

**Conclusão:** Mesmo sem conhecer a chave inicialmente, palavras repetidas permitem descobrir o deslocamento da Cifra de César.

# Criptoanálise da Cifra de César por Força Bruta

**Situação:** Se soubermos que o texto cifrado é uma cifra de César, podemos usar força bruta para descobrir o texto claro.

## Estratégia:

- Testar todas as 25 chaves possíveis ( $k = 1 \dots 25$ ).
- Observar qual saída produz texto inteligível.

## Exemplo:

- Texto cifrado: PHHW PH DIWHU WKH WRJD SDUWB
- Testando todas as chaves, o texto claro correto aparece na terceira tentativa: meet me after the toga party

## Condições que tornam a força bruta viável:

1. Algoritmos de encriptação e deciptação são conhecidos.
2. Espaço de chaves pequeno (apenas 25).
3. Linguagem do texto claro conhecida e reconhecível.

## Situação em redes e criptografia moderna:

- Algoritmos de encriptação geralmente são conhecidos.
- O espaço de chaves é enorme, tornando a força bruta impraticável.
  - Exemplo: Triple DES com chave de 168 bits  $\rightarrow 2^{168} \approx 3,7 \times 10^{50}$  chaves possíveis.
- Reconhecimento do texto claro pode ser difícil se:
  - A linguagem do texto não for conhecida.
  - O arquivo estiver compactado ou abreviado (ex.: ZIP).

**Conclusão:** A força bruta é viável apenas em cifras com pequeno espaço de chaves ou quando o texto claro é facilmente reconhecível, como na Cifra de César.

# Cifras Monoalfabéticas

**Definição:** Uma cifra monoalfabética usa uma permutação completa do alfabeto como chave. Cada letra do texto claro é substituída por uma letra correspondente no alfabeto cifrado.

**Exemplo:**

|                    |                                                     |
|--------------------|-----------------------------------------------------|
| <b>Texto claro</b> | A B C D E F G H I J K L M N O P Q R S T U V W X Y Z |
| <b>Chave</b>       | Q W E R T Y U I O P A S D F G H J K L Z X C V B N M |

**Observações:**

- A tabela de substituição (linha do texto cifrado) é a **chave** do sistema.
- Espaço de chave enorme:  $26! \approx 4 \times 10^{26}$  possibilidades.
- Muito mais seguro que a cifra de César contra ataques por força bruta.
- Cada letra do texto claro mapeia para exatamente uma letra do texto cifrado.

**Conclusão:** Em cifras monoalfabéticas, a chave é a própria permutação do alfabeto, garantindo um espaço de chave muito grande e maior resistência a ataques simples.

# Resumo da Lógica das Cifras Monoalfabéticas e César

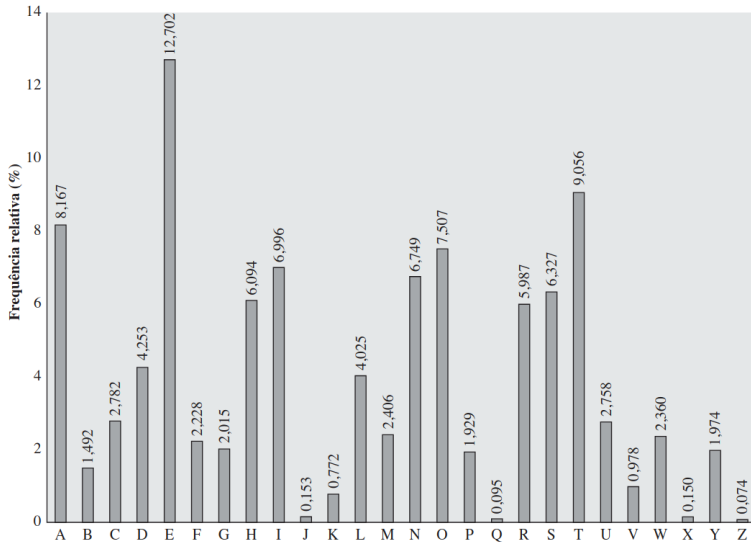
**O que é público:** O algoritmo, ou seja, como a substituição é feita. **O que é secreto:** A chave, que determina o mapeamento específico.

| Característica   | Cifra de César                | Substituição Monoalfabética                                                           |
|------------------|-------------------------------|---------------------------------------------------------------------------------------|
| Algoritmo        | Substituir por deslocamento   | Substituir por consulta a tabela                                                      |
| Chave            | Número $K$ (ex: 3)            | Tabela completa (ex: $A \rightarrow X$ , $B \rightarrow M$ , $C \rightarrow O$ , ...) |
| Espaço de chaves | Pequeno (25 chaves possíveis) | Enorme ( $26! \approx 4 \times 10^{26}$ )                                             |

**Observação:** A segurança de ambos depende do segredo da chave. Enquanto a Cifra de César é vulnerável à força bruta, a cifra monoalfabética é muito mais resistente, **mas ainda suscetível a ataques de análise de frequência.**

# O problema das cifras monoalfabéticas: a frequência das letras

**Figura 2.5** Frequência relativa de letras no texto em inglês.



# Cifras Monoalfabéticas com Homófonos

## **Problema das cifras monoalfabéticas:**

- Frequências de letras do alfabeto original permanecem no texto cifrado.
- Isso facilita ataques de análise de frequência.

## **Solução: Homófonos**

- Atribuir vários símbolos diferentes para a mesma letra do texto claro.
- Exemplo: a letra E poderia ser codificada como 16, 74, 35 ou 21, usados aleatoriamente ou em rodízio.
- Se o número de homófonos for proporcional à frequência da letra, a informação de frequência única é praticamente eliminada.

## **Observações:**

- Gauss acreditava ter criado uma cifra indecifrável usando homófonos.
- Mesmo com homófonos, padrões de múltiplas letras (digramas, trigramas) ainda podem ser explorados na criptoanálise.
- Portanto, ataques baseados em análise estatística ainda são possíveis.

# Importância dos Digramas na Criptoanálise

## Por que digramas importam?

- Análise de digramas detecta **pares de letras** que aparecem frequentemente em um idioma.
- Mesmo cifras monoalfabéticas com homófonos ainda deixam padrões de digramas visíveis.
- Cada idioma possui digramas e trigramas comuns; explorá-los ajuda a quebrar cifras.

**Exemplos de digramas comuns em português:** DE, ES, EN, NT, RE, RA, AR, OS, TE, CO

## Exemplo prático:

- Texto cifrado com substituição monoalfabética: símbolos %, &, \$# aparecem com frequência.
- Suspeita: % = E, & = A.
- Sequências %& e &% reforçam os digramas EA e AE.
- Sequência \$#% sugere "NTE", como em "mente", "gente", "frente".
- Padrões de digramas ajudam a deduzir a correspondência de símbolos e letras.

# Cifra Playfair

## Características principais:

- Cifra de múltiplas letras (substitui digramas, não letras isoladas).
- Cada par de letras no texto claro é convertido em um digrama no texto cifrado.
- Baseada em uma matriz  $5 \times 5$  de letras.

## Construção da matriz:

- Preencher a matriz com as letras da palavra-chave (sem duplicatas) da esquerda para a direita e de cima para baixo.
- Completar a matriz com as letras restantes do alfabeto.
- As letras I e J são consideradas uma só.

## Resumo do funcionamento:

- O texto claro é dividido em digramas (pares de letras).
- Cada digrama é substituído por outro digrama segundo regras da matriz.
- Essa abordagem dificulta a análise por frequência de letras individuais.

## Cifra playfair - Chave "monarchy"

|   |   |   |     |   |
|---|---|---|-----|---|
| M | O | N | A   | R |
| C | H | Y | B   | D |
| E | F | G | I/J | K |
| L | P | Q | S   | T |
| U | V | W | X   | Z |

# Cifra Playfair - Regras de Encriptação

**Criptografia por digramas:** O texto claro é encriptado duas letras de cada vez.

**Regras:**

1. **Letras repetidas em um par:** Inserir uma letra de preenchimento (ex.: 'x').  
Exemplo: balloon  $\rightarrow$  ba lx lo on.
2. **Letras na mesma linha da matriz:** Substituir cada letra pela à direita, de forma rotativa. Exemplo: ar  $\rightarrow$  RM.
3. **Letras na mesma coluna da matriz:** Substituir cada letra pela abaixo, de forma rotativa. Exemplo: mu  $\rightarrow$  CM.
4. **Caso geral (retângulo):** Cada letra do par é substituída pela letra na sua linha e na coluna da outra letra do par. Exemplo: hs  $\rightarrow$  BP, ea  $\rightarrow$  IM (ou JM).

# Cifra Playfair - Contexto e Segurança

## **Avanços sobre cifras monoalfabéticas:**

- Enquanto uma cifra monoalfabética lida com 26 letras, a Playfair trabalha com  $26 \times 26 = 676$  digramas.
- A análise de frequência torna-se muito mais difícil, pois os padrões de digramas são menos evidentes.
- Foi considerada indecifrável por muito tempo.

## **Uso histórico:**

- Sistema de campo padrão do Exército britânico na Primeira Guerra Mundial.
- Ainda usada pelo Exército dos EUA e forças aliadas na Segunda Guerra Mundial.

## **Limitações:**

- Apesar da aparente segurança, a Playfair ainda deixa rastros da estrutura da linguagem do texto claro.
- Algumas centenas de letras de texto cifrado geralmente são suficientes para quebrá-la.

# Cifras Polialfabéticas e Vigenère

## Cifras polialfabéticas:

- Melhoram as cifras monoalfabéticas usando diferentes substituições ao longo da mensagem.
- Um conjunto de regras de substituição monoalfabéticas é utilizado.
- Uma chave define qual regra é aplicada a cada posição da mensagem.

## Cifra de Vigenère:

- Uma das mais conhecidas e simples cifras polialfabéticas.
- Utiliza 26 cifras de César com deslocamentos de 0 a 25.
- Texto cifrado  $C = C_0, C_1, \dots, C_{n-1}$  é calculado como:

$$C_i = (p_i + k_j) \mod 26$$

onde  $p_i$  é a letra do texto claro e  $k_j$  é a letra da chave, repetindo a chave conforme necessário.

**Resumo:** Cada letra do texto claro é cifrada usando uma regra que varia ao longo da mensagem, definida pela chave.

# Cifra de Vigenère: Operação Detalhada

## Como funciona:

- Cada letra do texto claro é cifrada usando uma letra da chave correspondente.
- Para textos mais longos que a chave, a chave é repetida ciclicamente.

## Equação de encriptação:

$$C_i = (p_i + k_{i \bmod m}) \bmod 26$$

- $p_i$ : letra do texto claro na posição  $i$
- $k_{i \bmod m}$ : letra da chave correspondente, repetida quando necessário
- $m$ : comprimento da chave

## Equação de decifração:

$$p_i = (C_i - k_{i \bmod m}) \bmod 26$$

**Comparação com César:** Cada caractere do texto claro é cifrado com uma cifra de César diferente, determinada pela letra da chave.

# Cifra de Vigenère - Exemplo

- **Palavra-chave:** "deceptive"
- **Mensagem:** "we are discovered save yourself"

chave:                   deceptivedeceptivedeceptive  
texto claro:           wearediscoveredsaveyourself  
texto cifrado:        ZICVTWQNGRZGVTWAVZHCQYGLMGJ

Expresso numericamente, temos o seguinte resultado:

|               |    |   |   |    |    |    |    |    |   |    |    |   |    |    |
|---------------|----|---|---|----|----|----|----|----|---|----|----|---|----|----|
| chave         | 3  | 4 | 2 | 4  | 15 | 19 | 8  | 21 | 4 | 3  | 4  | 2 | 4  | 15 |
| texto claro   | 22 | 4 | 0 | 17 | 4  | 3  | 8  | 18 | 2 | 14 | 21 | 4 | 17 | 4  |
| texto cifrado | 25 | 8 | 2 | 21 | 19 | 22 | 16 | 13 | 6 | 17 | 25 | 6 | 21 | 19 |

# Vulnerabilidade da Cifra de Vigenère

**Ocultação da frequência de letras:** Cada letra do texto claro pode gerar múltiplas letras cifradas, dependendo da letra correspondente da chave, dificultando a análise de frequência simples.

**Informações ainda presentes:** Apesar da melhoria em relação à cifra Playfair, padrões do texto claro ainda podem aparecer. Por exemplo, sequências repetidas no texto cifrado podem indicar o comprimento da chave.

**Exemplo:** Um analista observa que a sequência “VTW” aparece duas vezes a uma distância de 9 caracteres. Isso sugere que a chave pode ter 3 ou 9 letras de extensão, embora a repetição possa ser casual.

**Ataque possível:** Se a mensagem for longa, múltiplas sequências repetidas permitem ao analista estimar o tamanho da chave. Esse é o princípio do ataque de Kasiski, usado para quebrar cifras polialfabéticas.

# Cifra de Vigenère com Auto-Chave

**Problema da repetição da chave:** A periodicidade da palavra-chave facilita ataques de criptoanálise.

**Solução proposta por Vigenère:** Usar uma palavra-chave concatenada ao próprio texto claro, formando uma *chave corrente* tão longa quanto a mensagem.

**Exemplo:**

- Chave: `deceptivewearediscoveredsav`
- Texto claro: `wearediscoveredsaveyourself`
- Texto cifrado: `ZICVTWQNGKZEIIGASXSTSLVVWLA`

**Observação:** Mesmo assim, o esquema ainda é vulnerável à criptoanálise estatística, pois a chave compartilha a distribuição de frequência do texto claro.

# Cifra de Vernam

**Ideia principal:** Usar uma palavra-chave tão longa quanto o texto claro, sem relação estatística com ele.

**Introdução:** Proposta em 1918 por *Gilbert Vernam*, engenheiro da AT&T. Opera sobre **dados binários** (bits), e não sobre letras.

**Definição:**

$$c_i = p_i \oplus k_i$$

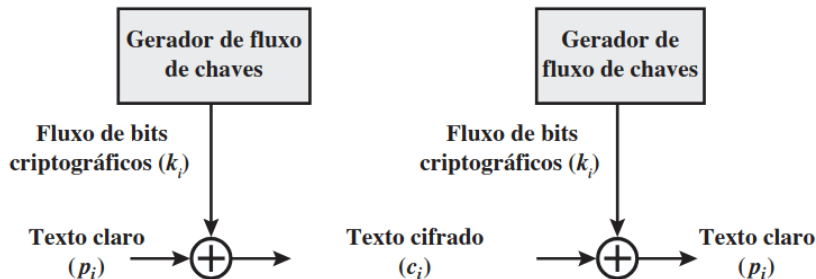
$$p_i = c_i \oplus k_i$$

onde:

- $p_i$  = bit do texto claro na posição  $i$ ,
- $k_i$  = bit da chave na posição  $i$ ,
- $c_i$  = bit do texto cifrado na posição  $i$ ,
- $\oplus$  = operação *XOR* (OU-exclusivo).

**Observação:** Vernam propôs uma chave muito longa, mas ainda repetida. Com texto cifrado suficiente, o sistema pode ser quebrado com análise estatística e *texto claro provável*.

**Figura 2.7** Cifra de Vernam.



## Melhoria proposta por Joseph Mauborgne (Exército dos EUA):

- Uso de uma **chave aleatória**, tão longa quanto a mensagem.
- A chave nunca é repetida.
- Cada mensagem exige uma nova chave, que deve ser descartada após o uso.

## Propriedades:

- O texto cifrado é totalmente aleatório.
- Não existe correlação estatística com o texto claro.
- **Inquebrável**: não há informação no texto cifrado que permita deduzir o texto claro sem a chave.

## Exemplo - One-time pad

ANKYODKYUREPFJBYOJDSPLREYIUNOFDOIUERFPLUYTS

Agora mostramos duas decifrações diferentes usando duas chaves distintas:

texto cifrado: ANKYODKYUREPFJBYOJDSPLREYIUNOFDOIUERFPLUYTS

chave: *pxlmvmsydofoyrvzwc tnlebnecvgdupahfzzlmnyih*

texto claro:\* mr mustard with the candlestick in the hall

texto cifrado: ANKYODKYUREPFJBYOJDSPLREYIUNOFDOIUERFPLUYTS

chave: *mfugpmiydgaxgoufhkl11lmhsqdgogtewbqfgyovuhwt*

texto claro:\*\* miss scarlet with the knife in the library

# One-Time Pad: Segurança e Limitações

## Segurança:

- A segurança decorre inteiramente da **aleatoriedade da chave**.
- Se a chave é verdadeiramente aleatória, o texto cifrado também será.
- Não existem padrões ou regularidades para um criptoanalista explorar.
- Representa a **cifra perfeita**: segurança completa em teoria.

## Limitações práticas:

1. Geração de grandes quantidades de números verdadeiramente aleatórios.
2. Distribuição e proteção das chaves — uma nova chave, do mesmo tamanho da mensagem, é necessária para cada comunicação.

## Segredo perfeito

Por causa dessas dificuldades, o one-time pad tem utilidade limitada, e aplicação principalmente para canais de pouca largura de banda que exigem segurança muito alta. O one-time pad é o único criptossistema que apresenta o que é conhecido como **segredo perfeito**.

# Aula 6 - Cifras de Fluxo

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação  
Universidade Federal do Rio de Janeiro  
gabrielaquino@ic.ufrj.br

Compilado em:  
April 3, 2026

# Princípio de Kerckhoffs

## Definição

O Princípio de Kerckhoffs, formulado por Auguste Kerckhoffs no século XIX, afirma que a segurança dos dados encriptados deve depender apenas da chave usada, mesmo que o método ou algoritmo seja de conhecimento público.

## Implicação

Esse princípio denuncia a falácia da *segurança pela obscuridade*, evidenciando que, se a segurança depende do segredo do método, então o método é falho.

- Uma **técnica de substituição** é aquela em que as letras do texto claro são substituídas por outras letras, números ou símbolos. Se o texto claro for visto como uma sequência de bits, então a substituição envolve trocar padrões de bits de texto claro por padrões de bits de texto cifrado.
- **Cifra de fluxo**: Encripta um fluxo de dados digital um bit ou um byte por vez.
- Exemplos clássicos:
  - Vigenère autochaveada
  - Vernam

## Da aula passada: Cifra de Fluxo: Vernam (One-Time Pad)

- Cada bit do texto claro é combinado com um bit da chave usando **XOR**.
- Se a chave for tão longa quanto a mensagem e aleatória, temos o **One-Time Pad**, inquebrável.
  - É uma versão ideal da cifra de Vernam.
  - O fluxo de chaves ( $k_i$ ) tem o mesmo tamanho do fluxo de bits do texto claro ( $p_i$ ).
  - Se a chave for realmente aleatória, a cifra é **inquebrável**.
  - Limitação: o fluxo de chaves precisa ser **pré-distribuído** de forma segura.
  - Problema prático: inviável para grandes volumes de tráfego devido à logística de distribuição.
- Se a chave for curta e repetida, a segurança é comprometida.

# XOR: Base das Cifras de Fluxo

## Tabela-verdade da função XOR

| A | B | S |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Onde  $S = A \oplus B$

## Exemplo de Encriptação e Decriptação

Para cifrar (c) =  $TEXTO \oplus CHAVE$ :

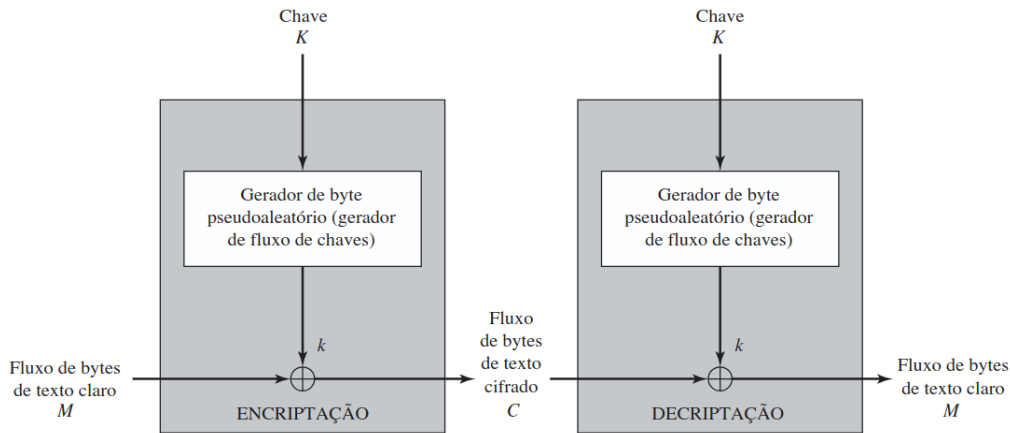
|                 |   |   |   |   |
|-----------------|---|---|---|---|
| Texto claro (p) | 1 | 0 | 1 | 1 |
| Chave (k)       | 0 | 1 | 0 | 1 |
| Cifrado (c)     | 1 | 1 | 1 | 0 |

Para decriptar (d):  $CIFRADO \oplus CHAVE$ :

|                 |   |   |   |   |
|-----------------|---|---|---|---|
| Cifrado (c)     | 1 | 1 | 1 | 0 |
| Chave (k)       | 0 | 1 | 0 | 1 |
| Texto claro (p) | 1 | 0 | 1 | 1 |

# Cifra de fluxo

**Figura 7.7** Diagrama da cifra de fluxo.



- Uma cifra de fluxo típica encripta um byte de texto claro por vez, embora possa operar sobre um bit ou unidades maiores.
- Uma chave é inserida em um gerador de bits pseudoaleatórios que produz um fluxo de números de 8 bits aparentemente aleatórios.
- A saída do gerador, chamada *fluxo de chaves*, é combinada byte a byte com o texto claro usando a operação **OU exclusivo (XOR)** bit a bit.

o byte de texto cifrado resultante será

$$\begin{array}{rcl} & 11001100 & \text{texto claro} \\ \oplus & \underline{01101100} & \text{fluxo de chaves} \\ & 10100000 & \text{texto cifrado} \end{array}$$

A deciptação requer o uso da mesma sequência pseudoaleatória:

$$\begin{array}{rcl} & 10100000 & \text{texto cifrado} \\ \oplus & \underline{01101100} & \text{fluxo de chaves} \\ & 11001100 & \text{texto claro} \end{array}$$

# Geradores de Fluxo em Cifras de Fluxo

- Por motivos práticos, o fluxo de bits é gerado por um **algoritmo** controlado por chave.
- O gerador deve produzir um fluxo **criptograficamente forte**.
  - Não deve ser possível prever partes futuras do fluxo a partir de partes já conhecidas.
- Os usuários compartilham apenas a **chave de geração**.
- Cada usuário pode então produzir localmente o mesmo fluxo de chaves.

## Semelhança com One-Time Pad:

- Uma cifra de fluxo também encripta um fluxo de bits de texto claro com um fluxo de chave.
- No one-time pad, o fluxo de chave é **verdadeiramente aleatório**.
- Na cifra de fluxo, o fluxo de chave é **pseudoaleatório**, gerado por um algoritmo.

## Considerações de projeto para cifras de fluxo:

1. A sequência de encriptação deverá ter um período grande?
2. O fluxo de chaves deverá se aproximar o máximo possível das propriedades de um fluxo de número aleatório verdadeiro?
3. A chave precisa ser suficientemente longa?

## Sequência de Encriptação:

- A sequência de bits gerada pelo gerador pseudoaleatório deve ter **um período longo**.
- O gerador produz um fluxo determinístico de bits que eventualmente se repete.
- Quanto maior o período, mais difícil é a criptoanálise.

## Relação com a cifra de Vigenère:

- Na Vigenère, uma palavra-chave curta se repete, facilitando ataques por análise de frequência.
- Na cifra de fluxo, um período maior no gerador pseudoaleatório reduz padrões repetitivos, aumentando a segurança.

## Requisitos para o fluxo de chaves em cifras de fluxo:

- O fluxo deve se aproximar de **um número aleatório verdadeiro**.
- Deve conter aproximadamente o mesmo número de 1s e 0s.
- Se o fluxo for tratado como bytes, os 256 valores possíveis devem aparecer com frequência aproximadamente igual.

## Impacto na segurança:

- Quanto mais aleatório o fluxo de chaves, mais aleatório será o texto cifrado.
- Isso dificulta a criptoanálise, pois padrões previsíveis são minimizados.

## Considerações sobre a chave:

- A saída do gerador de número pseudoaleatório depende da **chave de entrada**.
- Para proteção contra ataques de força bruta, a **chave deve ser suficientemente longa**.
- Assim como em cifras de bloco, recomenda-se **chaves de pelo menos 128 bits** com a tecnologia atual.

**Importância:** Uma chave curta compromete a segurança, mesmo que o gerador de fluxo seja forte.

# Comparação entre Cifras de Fluxo e de Bloco

## Segurança:

- Uma cifra de fluxo bem projetada pode ser tão segura quanto uma cifra de bloco com a mesma **tamanho de chave**.

## Vantagens das cifras de fluxo:

- Geralmente mais rápidas que cifras de bloco.
- Requerem menos código para implementação.
- Exemplo: RC4 pode ser implementado em poucas linhas.

## Atenção:

- Nos últimos anos, essa vantagem diminuiu com a introdução do AES, que é bastante eficiente em software.
- Técnicas de aceleração de hardware, como o AES Instruction Set da Intel, permitem executar uma rodada de encriptação/decriptação e geração de chave com grande velocidade.
- Ganhos de velocidade podem ser de cerca de uma ordem de grandeza em relação a implementações puramente em software.

# Vantagens e Riscos das Cifras de Bloco e Fluxo

## Cifras de Bloco:

- Permitem **reutilização de chaves** sem comprometer a segurança.

## Cifras de Fluxo:

- Se dois textos claros forem encriptados com a mesma chave, a criptoanálise torna-se trivial.
- O XOR dos dois textos cifrados produz o XOR dos textos claros originais.
- Vulnerável especialmente quando os textos claros possuem padrões conhecidos, como strings de texto, números de cartão de crédito ou outros fluxos de bytes estruturados.

# Escolha de Cifras: Fluxo vs. Bloco

## Cifras de Fluxo:

- Adequadas para **criptação/decriptação contínua** de dados.
- Ex.: canais de comunicação de dados, links de navegador/Web.

## Cifras de Bloco:

- Indicadas para **blocos de dados** inteiros.
- Ex.: transferência de arquivos, e-mails, bancos de dados.

**Observação:** Ambos os tipos podem ser usados em praticamente qualquer aplicação, dependendo do projeto.

## Descrição:

- Criada em 1987 por Ron Rivest para a RSA Security.
- Tamanho de chave **variável**, operações orientadas a **byte**.
- Baseada em uma **permutação aleatória**.

## Características:

- Período provavelmente maior que  $10^{100}$ .
- De 8 a 16 operações de máquina por byte de saída.
- Executa rapidamente em software.

## Hoje é considerado inseguro, mas ja teve uso:

- SSL/TLS (comunicação navegador-servidor Web)
- WEP e WPA (padrão IEEE 802.11 LAN sem fio)

## Histórico:

- Mantido como segredo comercial até 1994.
- Postado na Internet na lista Cypherpunks em setembro de 1994.

## RC4: Inicialização e Operação

### Inicialização:

- Chave de tamanho variável: 1 a 256 bytes (8 a 2048 bits).
- Vetor de estado  $S$  de 256 bytes:  $S[0], S[1], \dots, S[255]$ .
- $S$  contém uma **permutação de todos os números de 0 a 255**.

### Operação (Encriptação/Decriptação):

- Um byte  $k$  é gerado a partir de  $S$  selecionando uma entrada de forma sistemática.
- Cada vez que um byte  $k$  é gerado, os elementos de  $S$  são **novamente permutados**.
- A saída  $k$  é usada para XOR com o byte do texto claro/cifrado:

$$c_i = p_i \oplus k, \quad p_i = c_i \oplus k$$

- $p_i$  i-ésimo bit (ou byte) do texto claro.
- $k$  byte (ou bit) gerado pelo gerador de fluxo da cifra RC4.
- $c_i$  i-ésimo byte (ou bit) do texto cifrado,

# Inicialização do Vetor $S$ no RC4

- Defina o vetor de estado  $S$  com 256 elementos:

$$S[0] = 0, S[1] = 1, \dots, S[255] = 255$$

- Crie um vetor temporário  $T$  do mesmo tamanho que  $S$ .
- Preencha  $T$  usando a chave  $K$  de tamanho  $\text{keylen}$ :
  - Se  $\text{keylen} = 256$ , copie  $K$  diretamente para  $T$ .
  - Caso contrário, copie os primeiros  $\text{keylen}$  bytes de  $K$  para  $T$ , repetindo a chave tantas vezes quanto necessário até completar 256 elementos.

```
/* Inicialização */  
for  $i = 0$  to 255 do  
   $S[i] = i$ ;  
   $T[i] = K[i \bmod \text{keylen}]$ ;
```

## Exemplo - Inicialização do RC4 - Passo 1: Vetores S e T

**Chave:** 11111001 (1 byte)

**Vetor S:** Inicialmente com 256 elementos em ordem crescente:

$$S = [0, 1, 2, 3, \dots, 255]$$

**Vetor T:** Criado a partir da chave, repetida até ter 256 elementos:

$$T = [11111001, 11111001, 11111001, \dots](256 \text{ elementos})$$

**Resumo:**

- S contém todos os números de 0 a 255.
- T é preenchido sequencialmente com os bytes da chave, repetindo-a se necessário.

## RC4 - permutação

- Em seguida, usamos  $T$  para produzir a permutação inicial de  $S$ .
- Isso envolve começar com  $S[0]$  e ir até  $S[255]$ , e, para cada  $S[i]$ , trocar  $S[i]$  por outro byte em  $S$ , de acordo com um esquema ditado por  $T[i]$
- Como a única operação sobre  $S$  é uma troca, o único efeito é uma permutação.  $S$ , ainda, contém todos os números de 0 a 255.

```
/* Permutação inicial de S */
```

```
j = 0;
```

```
for i = 0 to 255 do
```

```
    j = (j + S[i] + T[i]) mod 256;
```

```
Swap (S[i], S[j]);
```

## RC4 - geração do fluxo

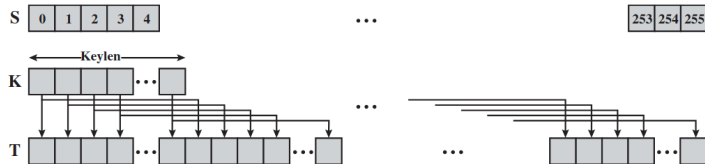
- Uma vez que o vetor S é inicializado, a chave de entrada não é mais usada.
- A geração de fluxo é percorrer todos os elementos de S[i] e, para cada S[i], trocar S[i] por outro byte em S de acordo com um esquema ditado pela configuração atual de S.
- Depois que S[255] é atingido, o processo continua, começando novamente em S[0]:

```
/* Geração de fluxo */  
i, j = 0;  
while (true)  
    i = (i + 1) mod 256;  
    j = (j + S[i]) mod 256;  
    Swap (S[i], S[j]);  
    t = (S[i] + S[j]) mod 256;  
    k = S[t];
```

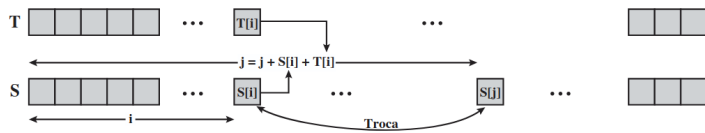
- Para encriptar, faça o XOR do valor k com o próximo byte do texto claro. Para decriptar, faça o XOR do valor k com o próximo byte do texto cifrado.

# RC4 - big picture

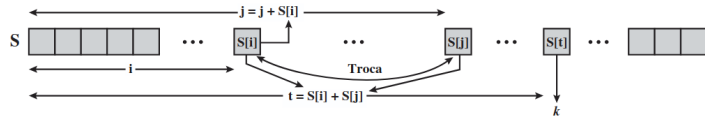
Figura 7.8 RC4.



(a) Estado inicial de  $S$  e  $T$



(b) Permutação inicial de  $S$



(c) Geração de fluxo

## **Segurança do RC4:**

- RC4 é resistente a ataques práticos se a chave for suficientemente longa (ex: 128 bits).
- Muitos estudos analisaram métodos de ataque, mas não são práticos para chaves de tamanho adequado.

## **Problema no WEP:**

- O protocolo WEP (para redes 802.11) mostrou-se vulnerável devido à forma de geração de chaves.
- O problema não está no RC4 em si, mas no uso incorreto da chave.
- O uso do WEP é desencorajado hoje em dia

# Aula 7 - Wired Equivalent Privacy (WEP)

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação  
Universidade Federal do Rio de Janeiro  
gabrielaquino@ic.ufrj.br

Compilado em:  
April 3, 2026

- Nos primeiros cinco anos do padrão IEEE 802.11, apenas um método de segurança foi definido: o **Wired Equivalent Privacy (WEP)**.
- Muitas vezes, WEP foi identificado incorretamente como *Wireless Effective Privacy* e outras variantes.
- Com o aumento da popularidade das redes Wi-Fi no ano 2000, a comunidade criptográfica começou a analisar o WEP e rapidamente encontrou vulnerabilidades.
- Já em 2001, ferramentas para quebrar o WEP em pouco tempo estavam disponíveis na Internet.

# Objetivos do WEP segundo o IEEE 802.11 (1999)

- **Força Razoável:**

- A segurança depende da dificuldade de descobrir a chave secreta por ataque de força bruta.
- Relaciona-se ao tamanho da chave e à frequência de troca das chaves e do vetor de inicialização (IV).

- **Exportabilidade:**

- Foi projetado de modo a facilitar a aprovação de exportação pelo Departamento de Comércio dos EUA.
- No entanto, devido ao clima político e legal da época, não havia garantias de exportabilidade.
- O padrão IEEE 802.11 especificava apenas o uso de chaves de 40 bits.
- **Problema:** 40 bits é um tamanho muito pequeno para resistir a ataques de força bruta, **o que explicava sua aceitação nas regras de exportação.**
- **Exemplo de ideia da época:** Se um banco fosse utilizar LAN sem fio, teria seu próprio protocolo de segurança rodando sobre o WEP.

# Objetivos do WEP segundo o IEEE 802.11 (1999)

- **Auto-Sincronização:**
  - O WEP é auto-sincronizável para cada mensagem
  - Propriedade importante em algoritmos de enlace de dados, onde a perda de pacotes pode ser alta.
  - Basicamente, isso significa que cada pacote deve ser encriptado separadamente, de forma que, dado um pacote e a chave, você tenha todas as informações necessárias para decriptá-lo.
  - Claramente, não se deseja uma situação em que a perda de um único pacote torne todos os pacotes seguintes indecifráveis.
- **Eficiência:** O algoritmo é eficiente, podendo ser implementado em **hardware ou software**.
- **Opcionalidade:** A implementação e o uso do WEP eram **opcionais** no padrão IEEE 802.11.

# Como o WEP foi "vendido" na época

- **Influência do Marketing:**

- Na promoção do IEEE 802.11, a palavra "razoável" foi omitida
- Na época WEP passou a ser descrito simplesmente como seguro.
- Após a flexibilização das restrições de exportação, fabricantes criaram extensões não padronizadas com chaves de 104 bits
- **Propaganda:** WEP era "extremamente" ou "absolutamente" seguro.

- **Efeito no Mercado** Essas extensões de chave foram incorporadas à especificação Wi-Fi e se tornaram o padrão da indústria em 1999. Na visão dos gestores de marketing, o WEP estava agora completamente seguro.

- **IMPORTANTE: Erro na Definição de Segurança:**

- Aceitar o conceito de um nível "razoável" de segurança foi um erro.
- Só existem dois tipos de segurança: forte ou nenhuma.
- O padrão deveria: (i) OU ter incorporado uma solução realmente robusta OU deixado claro que a segurança deveria ser provida por outros meios (ex. VPN, HTTPS..).

- Alguns críticos acusam os projetistas do padrão IEEE 802.11 original por criarem o WEP com fraquezas inerentes.
- No entanto, é importante lembrar que, na época, o WEP **não foi projetado para prover segurança em nível militar**.
- O objetivo era fornecer uma proteção semelhante à de uma rede cabeada: **difícil, mas não impossível de quebrar**.

- Para muitos usuários, o WEP foi a única escolha até a adoção de novos métodos de segurança no padrão IEEE 802.11.
- Mesmo com suas fraquezas, o WEP era considerado melhor do que não ter nenhuma proteção.
- O WEP cria uma **barreira mínima**, suficiente para desencorajar atacantes casuais, que preferem procurar redes totalmente abertas.
- **Eficácia do WEP em Ambientes Domésticos:**
  - A maioria dos ataques ao WEP depende da coleta de um número significativo de pacotes transmitidos.
  - Em redes domésticas, onde o tráfego seria a ser pequeno, o WEP **poderia oferecer** um nível razoável de segurança.

- **Fases da segurança no WEP:**

- Autenticação: o dispositivo prova sua identidade ao ponto de acesso.
- Criptografia: garante a confidencialidade das mensagens após a autenticação.

- **Propósito da autenticação:**

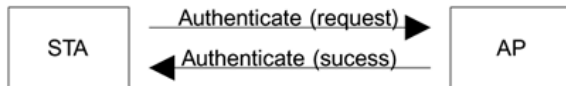
- Permitir que cada parte prove que é quem afirma ser.
- Similar a assinar um cheque: confirma a identidade do remetente.

- **Propósito da criptografia:**

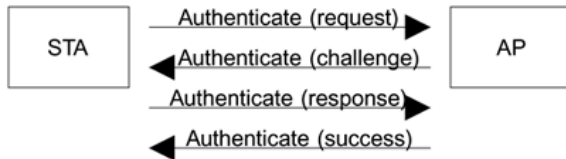
- Permitir que os dados trafegados não sejam vistos por quem está interceptando (Confidencialidade)
- Fazer com que quem envie ou receba dados na rede faça-o se possuir a chave.

# Autenticação no WEP

- A autenticação WEP tem o objetivo de provar ao ponto de acesso legítimo que o dispositivo móvel conhece a chave secreta.



Open Authentication



WEP Authentication

# Formato da Mensagem de Autenticação (802.11)

## Campos principais

- **Algorithm Number:** Indica o tipo de autenticação
  - 0 – Open System
  - 1 – Shared Key (WEP)
- **Transaction Sequence:** Número da etapa na sequência de autenticação
  - Mensagem 1, Mensagem 2, (e Mensagem 3 no caso do WEP)
- **Status Code:** Indica sucesso ou falha na autenticação (última mensagem)
- **Challenge Text:** Usado apenas na autenticação por chave compartilhada (WEP)

**Figure 6.2. Authentication Message Format**

|               |                  |             |                |
|---------------|------------------|-------------|----------------|
| Algorithm Num | Transaction Seq. | Status Code | Challenge Text |
|---------------|------------------|-------------|----------------|

## Definição

Processo de autenticação que verifica uma identidade exigindo que uma resposta correta seja fornecida a um **desafio** (*challenge*).

## Como funciona em sistemas computacionais

- O sistema envia um valor imprevisível (desafio).
- O usuário deve calcular e enviar a resposta correta.
- A resposta é validada para confirmar a identidade.

# Autenticação WEP - Mecanismo de Challenge-Response

- Quando o dispositivo solicita autenticação, o ponto de acesso envia um número aleatório chamado **challenge text**.
  - Um número arbitrário de 128 bits, preferencialmente aleatório.
- O dispositivo móvel então criptografa esse número com a chave secreta usando WEP e envia de volta ao ponto de acesso.
- Como o ponto de acesso lembra o número aleatório enviado, ele pode verificar se a resposta foi criptografada com a chave correta.
- O dispositivo deve conhecer a chave para criptografar o valor aleatório corretamente.

# Limitações da Autenticação no 802.11

## Observações importantes:

- Isso não prova ao dispositivo móvel que o ponto de acesso conhece a chave.
- Se um atacante estiver escutando, ele terá um par de *plaintext-ciphertext* para começar ataques.
- Devido a essas vulnerabilidades, a Wi-Fi Alliance abandonou o uso desse mecanismo de autenticação.

## Problemas

- O método de autenticação com WEP é praticamente inútil.
- Na verdade, é pior que inútil, pois fornece ao atacante informações úteis para explorar.

# WEP e o Processo de Autenticação

- Quando o WEP está habilitado, as mensagens de dados são criptografadas.
- Objetivo: impedir que atacantes entendam o conteúdo mesmo escutando o tráfego.
- O padrão IEEE 802.11 original previa duas fases:
  1. Autenticação.
  2. Criptografia dos dados.
- Para decodificar, é necessário conhecer a **chave secreta**.

# RC4 no WEP

- O WEP utiliza a cifra de fluxo **RC4** (Schneier, 1996).
- Funcionamento em alto nível:
  - Entrada: um byte do fluxo de dados por vez.
  - Saída: um byte diferente, aparentemente aleatório.
  - Resultado: a saída parece uma sequência de números aleatórios.
- A descriptografia é o processo inverso.
- O mesmo segredo é usado tanto para criptografar quanto para descriptografar (cifra **simétrica**).

**Figure 6.3. Stream Cipher**

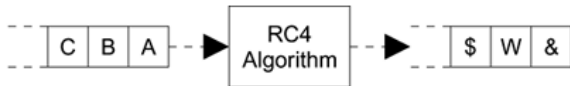


Figure: RC4 como cifra de fluxo no WEP

## RC4 e sua utilização no WEP

- RC4 é simples de implementar, sem operações complexas como multiplicação.
- Possui duas fases principais:
  - **Inicialização**: tabelas internas são construídas a partir da chave.
  - **Encriptação**: os dados passam pelo algoritmo e são cifrados.
- **No WEP, ambas as fases ocorrem a cada pacote transmitido.**
- Essa abordagem garante que a perda de um pacote não comprometa os seguintes.
- Contudo, também introduz vulnerabilidades exploradas em ataques posteriores.
  - **Vocês podem imaginar quais seriam?**

# Problema de usar chave fixa no WEP

- O WEP utiliza o algoritmo RC4.
- Com uma chave fixa: todos os pacotes seriam criptografados com a mesma chave.
- Consequência: textos repetidos geram sempre a mesma saída.
- Isso permite a um atacante identificar padrões, como endereços IP, já que certas partes dos pacotes se repetem.

## Exemplo do Problema

- Texto: "Source Address"
- Resultado cifrado (exemplo): "b%fP\*aF\$!Y"
- Sempre que o mesmo texto for cifrado com a mesma chave, o resultado será idêntico.
- Isso revela informação valiosa para o atacante.

## Solução: Initialization Vector (IV)

- Em vez de usar apenas a chave fixa, o WEP adiciona um número de 24 bits chamado **IV**.
- O IV muda a cada pacote transmitido.
- A chave efetiva passa a ser: **Chave Secreta (104 bits) + IV (24 bits)**.
- Isso garante que pacotes iguais gerem resultados diferentes.

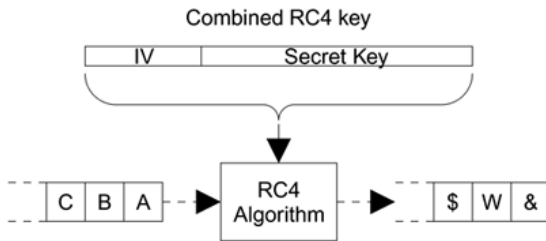
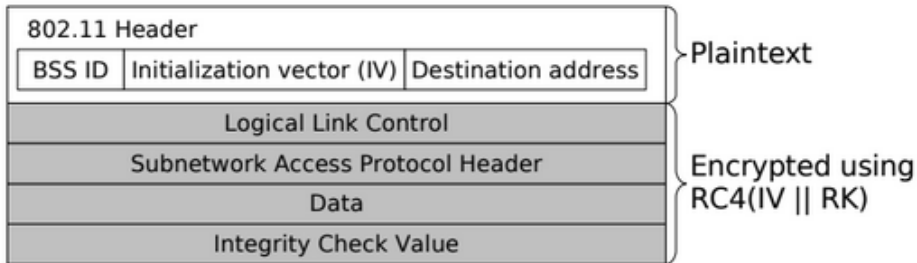


Figure: RC4 com IV e Secret Key

## Limitação do IV

- O IV é transmitido junto com o pacote, logo não é secreto.
- Chamar isso de “128-bit security” é enganoso, pois apenas 104 bits são realmente secretos.
- **Mesmo assim...** o uso do IV **dificulta** a identificação de padrões simples.

**Fig. 1.** A 802.11 frame encrypted using WEP



# Problemas do Vetor de Inicialização (IV) no WEP

- O IV não é secreto: é transmitido abertamente junto com os pacotes.
- **O mesmo IV nunca deveria ser reutilizado com a mesma chave secreta.**
- No WEP, o IV tem apenas **24 bits**  $\Rightarrow$  cerca de 17 milhões de valores possíveis.
- Em um ponto de acesso a 11 Mbps:
  - todos os IVs podem ser usados em poucas horas.
- Reutilização de IVs é inevitável, pois as chaves raramente são trocadas.

## Problemas adicionais:

- Muitos dispositivos reiniciam sempre com o mesmo IV (**Problema de implementação**).
- Sequências “pseudoaleatórias” de IV podem se repetir em diferentes dispositivos.
- Consequência: IVs repetidos facilitam ataques contra o WEP.

# Funcionamento do WEP e Fragmentação

- Aplicações enviam os dados.
- Os dados pode ser dividido em vários fragmentos.
- Cada fragmento é transformado em um **MPDU** (MAC Protocol Data Unit).
- Para cada MPDU:
  - É adicionado um cabeçalho MAC.
  - É incluído um **ICV** no final.
  - Cada fragmento é processado separadamente pelo WEP.
- O processo de criptografia trata o MPDU como um bloco de bytes (10 a 1500 bytes).
- Antes da cifragem, adiciona-se o **Integrity Check Value (ICV)**.

# Integrity Check Value (ICV)

- Objetivo: detectar alterações nos dados durante a transmissão.
- Calculado como um CRC (Cyclic Redundancy Check) de 4 bytes.
- O ICV é adicionado **antes da criptografia** WEP.
- O CRC convencional ainda é adicionado **após a criptografia**.
- Ideia: como o ICV é cifrado, um atacante não poderia alterá-lo.

## Limitações do ICV

- Funciona contra **erros acidentais**, mas não contra ataques ativos.
- Atacantes podem recalcular o CRC após modificar a mensagem.
- Em teoria, o ICV deveria tornar a mensagem *à prova de adulteração*.
- Na prática, há como contornar essa proteção.

# Preparando o frame para transmissão

- Pega os dados e calcula o ICV
- Após o ICV ser anexado, o quadro está pronto para criptografia.
- Seleciona-se um valor de **IV** (24 bits) e o anexa à chave WEP.
- O motor do **RC4** é inicializado.
- Cada byte do bloco (Dados + ICV) é processado → sai um byte criptografado.
- O quadro transmitido contém:
  - IV (3 bytes) + KeyID (1 byte) no início.
  - Dados criptografados + ICV criptografado.
  - Cabeçalho MAC e CRC ao final.
- Um bit no cabeçalho MAC indica que o quadro está protegido por WEP.

# Recebendo o frame WEP

- O receptor identifica o **bit WEP** no cabeçalho.
- Lê o valor do IV (3 bytes) e o KeyID (1 byte).
- Seleciona a chave WEP correta e inicializa o RC4.
- Processa cada byte do fluxo criptografado → obtém os dados originais.
- Como RC4 é simétrico, criptografia = decifração (dupla aplicação anula o efeito).
- Calcula o ICV e compara com o recebido.
- Se válido, os dados são entregues para processamento superior.

# Falhas do WEP - Autenticação

- Autenticação serve para provar a identidade; idealmente, cada comunicação deve ser autenticada.
- Autenticação mútua é necessária em Wi-Fi:
  - O usuário precisa verificar a rede.
  - A rede precisa verificar o usuário.
- Regras recomendadas para autenticação:
  1. Método robusto que não possa ser falsificado.
  2. Identidade deve persistir em transações subsequentes e não ser transferível.
  3. Autenticação mútua.
  4. Chaves de autenticação devem ser independentes das chaves de criptografia.

# Falhas na autenticação WEP

## Mecanismo: Challenge-response

- 1. O AP envia uma string aleatória de números.
- 2. A estação móvel cifra a string e envia de volta.
- 3. O AP decifra e compara com a string original.

## Problemas:

- **Regra 4 quebrada:** A chave usada na autenticação é a mesma usada para criptografia.
- **Regra 3 quebrada:** O AP não é autenticado; um AP malicioso pode enviar sucesso sem conhecer a chave.
- **Regra 2 quebrada:** Não há token para validar transações futuras.
- **Regra 1 irrelevante:** Devido às falhas anteriores, a robustez da autenticação é comprometida.

# Ataque à autenticação WEP via XOR

**Cenário:** Durante a autenticação, o AP envia um desafio de 128 bytes e a estação móvel responde cifrando com RC4.

**Transação capturada pelo atacante:**

Desafio (P) → Texto claro

Resposta (C) → Texto cifrado (RC4)

**Extraindo o key stream:**

$$R = C \oplus P$$

Porque  $P \oplus R = C$  e  $C \oplus P = R$

**Consequência:** O atacante agora conhece o key stream  $R$  correspondente ao IV usado. Ele pode reutilizá-lo em futuras autenticações, conseguindo se autenticar sem conhecer a chave secreta.

**Observação:** Essa falha fornece gratuitamente ao atacante os primeiros 128 bytes do key stream, que são os mais vulneráveis.

# Ataque à Autenticação por Challenge-Response

**Contexto:** O mecanismo de autenticação do WEP usava um desafio (*challenge*) que era criptografado pela estação e enviado de volta ao ponto de acesso.

**Cenário:** Um atacante captura a seguinte troca de mensagens durante uma autenticação WEP:

- Desafio (P) enviado pelo AP: 2A7F9C4E (hexadecimal)
- Resposta (C) enviada pela estação: D3B1AC8B (hexadecimal)

**Pergunta:**

1. Utilizando a operação XOR, calcule o keystream (R) usado pela estação para criptografar o desafio.
2. Explique como um atacante, conhecendo este par (P, C), pode se autenticar fraudulentamente na rede no futuro, sem precisar conhecer a chave secreta WEP.

## Cálculo do keystream (R) usando XOR

$$R = P \oplus C$$

$$R = 2A7F9C4E \oplus D3B1AC8B = F9CE30C5$$

2A7F9C4E = 0010.1010.0111.1111.1001.1100.0100.1110

D3B1AC8B = 1101.0011.1011.0001.1010.1100.1000.1011

---

F9CE30C5 = 1111.1001.1100.1110.0011.0000.1100.0101

**Atenção, para não confundir!**

Esse R não é a chave secreta da rede WEP, mas sim o keystream (fluxo de chave) que o algoritmo RC4 gerou naquele momento específico, usando a combinação do Vetor de Inicialização (IV) e da chave secreta.

- O atacante agora conhece o **keystream** correspondente ao IV usado nesta autenticação.
- Ele pode reutilizar o keystream com o mesmo IV em futuras autenticações.
- Consequência: consegue se autenticar **sem precisar conhecer a chave secreta WEP**.
- Esse é o ponto inicial para ataques de quebra de WEP.

# Problema da persistência da autenticação no WEP

- A autenticação acontece apenas no início, durante o challenge-response.
- Depois disso, o sistema não emite nenhum tipo de “token” ou prova de identidade que o usuário possa apresentar nas transações subsequentes.
- Então, para todo o resto da comunicação, a rede não revalida a identidade do usuário, e tudo se baseia apenas na chave de criptografia usada para cifrar/decriptar os pacotes.

# Replay Attack e WEP

## Exemplo de ataque:

- Um invasor captura quadros (frames) entre um AP e um dispositivo móvel usando um sniffer.
- Observa mensagens criptografadas e seu tamanho, sem decifrar o conteúdo.
- Quando a usuária legítima se desconecta, o invasor se conecta usando o MAC da vítima.
- O invasor reenvia (replay) uma mensagem previamente capturada.
- O AP aceita a mensagem, passando-a ao servidor, que autentica o invasor sem perceber a fraude.

## Problema:

- WEP não possui proteção contra replay.
- Sequência de número no MAC não é protegida pelo WEP.
- Mensagens antigas podem ser reenviadas sem precisar quebrar a criptografia.
- Consequência: invasor consegue se autenticar indevidamente.

# Falha do WEP na Detecção de Modificação de Mensagens

**Objetivo do WEP:** prevenir alterações nos dados transmitidos usando o *Integrity Check Value (ICV)*.

**Como funciona:**

- O ICV é um valor CRC de 32 bits calculado sobre os dados.
- ICV é adicionado ao final da mensagem e toda a sequência é criptografada.
- Teoricamente, alterações no ciphertext seriam detectadas na deciptação.

**Problema:**

- O CRC é linear: é possível prever como o ICV muda ao alterar bits da mensagem.
- A operação XOR usada na criptografia permite *bit flipping*: alterar bits no ciphertext altera os mesmos bits na mensagem decifrada.
- Um atacante que conheça os bits certos pode modificar a mensagem e ajustar o ICV correspondente, mantendo a integridade aparente.

**Conclusão:** WEP não fornece proteção efetiva contra modificações no ciphertext.

# Por que o IV é importante no WEP?

## **Função do IV (Initialization Vector):**

- Adicionado à chave secreta para formar a chave de criptografia usada pelo RC4.
- Garante que duas mensagens idênticas não gerem o mesmo ciphertext.

## **E se não houvesse IV?**

- RC4 seria inicializado sempre no mesmo estado.
- O fluxo de chave (key stream) seria idêntico para todos os frames.
- Consequência: um atacante que descubra o fluxo de chave de um frame pode descriptografar todos os outros frames.
- Não seria necessário conhecer a chave secreta!

## IV diferente para cada frame

### Por que usar IV diferente em cada frame?

- Adicionando o IV à chave secreta, o RC4 é inicializado em um estado diferente a cada frame.
- Consequência: o fluxo de chave (*key stream*) é diferente para cada criptografia.

### IV constante ou reutilizado é problemático:

- IV constante → mesmo problema do caso de chave fixa.
- IV reutilizado → permite que atacantes detectem padrões e quebrem a criptografia.
- Reuso inevitável se o número de IVs possíveis é limitado (24 bits no WEP).
- Seleção aleatória de IVs aumenta o risco de repetição cedo (paradoxo do aniversário).

## Paradoxo do Aniversário:

- Probabilidade de pelo menos duas pessoas compartilharem o mesmo aniversário.
- Surpreendentemente, com apenas 23 pessoas, a probabilidade  $> 50\%$ .
- Explicação: existem  $23 \times 22/2 = 253$  pares possíveis.

## Aplicação em criptografia:

- Ataques de colisão (*birthday attack*) em funções hash.
- Similarmente, reuso de IVs em WEP aumenta a chance de colisões no fluxo de chave.

**Lição para WEP:** O número limitado de IVs (24 bits) faz com que colisões aconteçam cedo, tornando o sistema vulnerável.

**Problema:** IVs de 24 bits são limitados e podem se repetir rapidamente.

**Solução ideal:** incrementar o IV a cada quadro enviado para maximizar o tempo até uma repetição.

**Exemplo prático:**

- IV de 24 bits  $\rightarrow$  16.777.216 valores possíveis.
- IEEE 802.11b transmite 500 quadros/s.
- Espaço de IVs esgotado em cerca de 7 horas.

**Problemas reais:**

- Vários dispositivos usando o mesmo IV/ chave  $\rightarrow$  colisões mais cedo.
- Erros de implementação: alguns fabricantes reiniciam o IV em 0 após reinício.
- Resultado: repetição de IV é comum e vulnerabilidade explorável.